Advanced React Web Development — The Complete Guide

Quick reference: 20. Cheat sheet at the end of this guide.

0. How to use this guide

This guide takes an advanced JavaScript developer to advanced-level React: how React works internally, which patterns teams use in production today, and how to ship a real app end to end. Part of a series (React, React Native, Next.js, Electron, Backend, DevOps, Freelancing, game development).

Assumed: closures, the event loop, promises/async, modules, prototypes, immutability, array methods, basic TypeScript.

Versions referenced (mid-2026 baseline — always check release notes): React 19.2+, React Compiler 1.x, TypeScript 5.x, Vite 7+, React Router 7, TanStack Query 5, Zustand 5, Redux Toolkit 2, Tailwind CSS 4, Vitest 3+, Playwright, ESLint 9 flat config.

0.1 How to read the code in this guide

- Every code block is annotated: `//` comments explain *why* a line exists, not just what it does.
- After important blocks, a **How it works** list walks through the moving parts in order.
- `// ❌` marks an anti-pattern you'll meet in real codebases; `// ✅` marks the fix.
- Snippets are excerpts: imports are omitted when they're obvious (`useState` from `react`, `z` from `zod`).
- The last chapter is a cheat sheet with the most-used code in one place.

0.2 The 2026 React landscape

Concern	Default choice	Common alternatives
SPA build tool	Vite	Rsbuild, Parcel
Full-stack framework	Next.js (App Router)	React Router 7 framework mode, TanStack Start

Concern	Default choice	Common alternatives
Language	TypeScript (strict)	—
Server state	TanStack Query	SWR, RTK Query, framework loaders
Client state	useState/useReducer + Zustand	Redux Toolkit, Jotai, XState
Routing (SPA)	React Router 7	TanStack Router
Forms	React Hook Form + Zod, or React 19 actions	TanStack Form, Conform
Styling	Tailwind v4 + shadcn/ui	CSS Modules, vanilla-extract, Panda
Testing	Vitest + Testing Library + MSW + Playwright	Jest, Cypress
Memoization	React Compiler (automatic)	manual useMemo/useCallback

0.3 What separates an advanced React developer

- Knows *why* a component re-renders and can predict it without the profiler.
- Treats effects as synchronization with external systems, not lifecycle hooks.
- Separates server state, client state, URL state and form state, and picks a tool per category.
- Designs component APIs (composition, compound components, headless hooks) others enjoy using.
- Owns quality: types, tests, accessibility, performance budgets, error handling, observability.
- Makes trade-off decisions (SPA vs SSR, library vs custom) and can defend them.

0.4 Mental model to carry through every chapter

```
// the screen is a pure function of the current state
UI = f(state)

render phase
  → React calls your components, builds a tree of
    elements, diffs it with the previous tree.
  → Must be PURE: no DOM writes, no network calls,
    no mutation of outside variables.
  → Can be paused, restarted or thrown away
    (concurrent rendering).

commit phase
  → React applies the minimal DOM changes, runs layout
    effects, the browser paints, then passive effects
    (useEffect) run. Never interrupted.
```

Everything — the Rules of Hooks, StrictMode's double rendering, concurrent features, the React Compiler — follows from keeping the render phase pure.

1. Project setup

A production SPA starts with Vite + React + TypeScript strict, path aliases, ESLint flat config with the hooks/compiler rules, Prettier, and a feature-based folder layout. Create React App is deprecated; do not use it.

1.1 Scaffold

```
# Create a Vite project from the official React + TypeScript
# template
npm create vite@latest acme-web -- --template react-ts
cd acme-web
# install the template's dependencies
npm i

# Dev-only tools: compiler plugin, lint rules, formatter,
# test stack, API mocking
npm i -D babel-plugin-react-compiler \
  eslint-plugin-react-hooks@latest \
  prettier prettier-plugin-tailwindcss vitest \
  @testing-library/react @testing-library/user-event \
  @testing-library/jest-dom jsdom msw

# Runtime libraries used throughout this guide
npm i react-router @tanstack/react-query zustand zod \
  react-hook-form @hookform/resolvers
```

1.2 vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';
import path from 'node:path';

export default defineConfig({
  plugins: [
    // The React plugin runs Babel on your components; we add
    // the React Compiler as a Babel plugin so memoization is
    // inserted automatically at build time (chapter 6).
    react({
      babel: { plugins: [['babel-plugin-react-compiler', {}]] },
    }),
    // Tailwind v4: no config file needed, scans your sources
    tailwindcss(),
  ],
  resolve: {
    // "@/features/auth" instead of "../../../features/auth" —
    // must match tsconfig "paths"
```

```
    alias: { '@': path.resolve(__dirname, 'src') },
  },
  server: {
    port: 5173,
    // During development, requests to /api/* are forwarded to
    // the backend on :3000. The browser sees one origin → no
    // CORS setup needed and cookies work as in production.
    proxy: {
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true,
      },
    },
  },
  build: {
    // upload to Sentry; don't serve them publicly
    sourcemap: true,
    rollupOptions: {
      // Split rarely-changing vendor code into its own chunks
      // so browsers keep them cached across deploys (your app
      // code changes often; React doesn't).
      output: {
        manualChunks: {
          react: ['react', 'react-dom'],
          router: ['react-router'],
        },
      },
    },
  },
  test: {
    // fake DOM for component tests
    environment: 'jsdom',
    // runs before every test file (matchers, MSW)
    setupFiles: './src/test/setup.ts',
    // describe/it/expect without imports
    globals: true,
  },
});
```

1.3 tsconfig essentials

```
{
  "compilerOptions": {
    // modern syntax; Vite handles browser compatibility
    "target": "ES2022",
    "lib": ["ES2023", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    // resolve imports the way Vite does
    "moduleResolution": "bundler",
    // new JSX transform: no `import React` needed
    "jsx": "react-jsx",
    // null checks, no implicit any, etc.
    "strict": true,
    // arr[0] is T | undefined → forces you to handle missing
    // items
    "noUncheckedIndexedAccess": true,
    // `{ a?: string }` can't be set to undefined explicitly
    "exactOptionalPropertyTypes": true,
    "noImplicitOverride": true,
    // type-only imports must use `import type` → cleaner
    // bundles
    "verbatimModuleSyntax": true,
    // don't type-check node_modules (speed)
    "skipLibCheck": true,
    "baseUrl": ".",
    // same alias as Vite
    "paths": { "@/*": ["src/*"] }
  },
  "include": ["src"]
}
```

1.4 ESLint 9 flat config

```
// eslint.config.js — "flat config": one array of config
// objects, applied in order
import js from '@eslint/js';
import tseslint from 'typescript-eslint';
import reactHooks from 'eslint-plugin-react-hooks';
import jsxA11y from 'eslint-plugin-jsx-a11y';

export default tseslint.config(
```

```
  // never lint build output
  { ignores: ['dist'] },
  // core JS rules
  js.configs.recommended,
  // TS rules that use type information
  ...tseslint.configs.strictTypeChecked,
  // Rules of Hooks + exhaustive deps + React Compiler
  // diagnostics
  reactHooks.configs['recommended-latest'],
  // accessibility mistakes in JSX
  jsxA11y.flatConfigs.recommended,
  {
    // lets type-aware rules find tsconfig
    languageOptions: {
      parserOptions: { projectService: true },
    },
    rules: {
      // forgot to await/handle a promise
      '@typescript-eslint/no-floating-promises': 'error',
      // `import type { X }`
      '@typescript-eslint/consistent-type-imports': 'error',
    },
  },
);
```

1.5 Folder architecture (feature-sliced, scales to 100k+ LOC)

```
src/
  app/       # bootstrapping only: providers, router, styles
    providers.tsx
    router.tsx
  features/  # vertical slices: each owns its UI, hooks,
             # API calls and state
    auth/
      api.ts       # fetch functions + query keys
      hooks.ts     # useLogin, useSession
      components/
      store.ts
      index.ts     # the feature's PUBLIC API: other code
                   # imports only from here
    projects/
  shared/    # domain-agnostic building blocks
```

```
    ui/      # Button, Dialog, Input (design system)
    lib/     # http client, formatters, utils
    hooks/   # useDebounce, useMediaQuery
  pages/     # thin route components composing features
  test/
```

Rules: features never import other features' internals (enforce with `eslint-plugin-boundaries` or `import/no-restricted-paths`); `shared` never imports from `features`. Why: when a feature is deleted or rewritten, nothing else breaks.

1.6 Environment variables

```
// src/shared/lib/env.ts
// Vite exposes ONLY variables prefixed with VITE_ to client
// code, by inlining them at build time. Everything in the
// bundle is public — never put secrets here.
import { z } from 'zod';

const schema = z.object({
  // must be a valid URL or the app refuses to start
  VITE_API_URL: z.string().url(),
  VITE_SENTRY_DSN: z.string().optional(),
});

// Parse once at startup: a missing or malformed variable fails
// loudly in development instead of causing a confusing runtime
// error deep inside the app.
export const env = schema.parse(import.meta.env);
```

```
// src/vite-env.d.ts — teaches TypeScript which env vars exist
// (autocomplete + type checks)
interface ImportMetaEnv {
  readonly VITE_API_URL: string;
  readonly VITE_SENTRY_DSN?: string;
}
```

1.7 Entry point and providers

```
// src/main.tsx — the first module the browser runs
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Providers } from '@/app/providers';
import { AppRouter } from '@/app/router';
import './index.css';

createRoot(document.getElementById('root')!, {
  // React 19 root-level error hooks: called for errors no
  // boundary caught (uncaught) and for errors a boundary
  // handled (caught). Send both to your error tracker.
  onUncaughtError: (err, info) =>
    reportError(err, info.componentStack),
  onCaughtError: (err, info) =>
    reportError(err, info.componentStack),
}).render(
  // StrictMode (dev only) double-invokes renders/effects to
  // expose impure code — keep it on.
  <StrictMode>
    <Providers>
      <AppRouter />
    </Providers>
  </StrictMode>,
);
```

```
// src/app/providers.tsx — every app-wide context in one place
// Created at module scope: ONE cache for the app's lifetime
// (creating it inside the component would make a new, empty
// cache on every render).
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      // data counts as fresh for 30 s → no refetch on every
      // mount
      staleTime: 30_000,
      // retry a failed request once before showing an error
      retry: 1,
      // opinion: avoid surprise refetches when switching tabs
      refetchOnWindowFocus: false,
    },
  },
```

```
});

export function Providers({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <QueryClientProvider client={queryClient}>
      <ThemeProvider>{children}</ThemeProvider>
      {/* Devtools only in development; tree-shaken from */}
      {/* production builds */}
      {import.meta.env.DEV && <ReactQueryDevtools />}
    </QueryClientProvider>
  );
}
```

1.8 package.json scripts

```
{
  "scripts": {
    "dev": "vite",
    "//dev": "dev server with hot module replacement",
    "build": "tsc -b && vite build",
    "//build": "type-check first; a type error fails it",
    "preview": "vite preview",
    "//preview": "serve the production build locally",
    "lint": "eslint . --max-warnings 0",
    "//lint": "warnings count as failures in CI",
    "typecheck": "tsc -b --noEmit",
    "test": "vitest",
    "test:e2e": "playwright test",
    "format": "prettier --write ."
  }
}
```

(JSON has no comments; the `"//name"` keys above are a common convention for notes — npm ignores them.) Add Husky + lint-staged to run `eslint --fix` and `prettier` on staged files at commit time, and run `typecheck`, `lint` and `test` in CI on every pull request.

2. The rendering model, internally

A component re-renders when **its state changes**, **its parent re-renders**, or **a context it reads changes**. Props changing is not a trigger by itself — props only change because a parent re-rendered. Master this and most performance questions answer themselves.

2.1 Elements are plain objects

```
const el = <Button kind="primary">Save</Button>;

// The JSX above is syntax sugar. The compiler turns it into
// a function call:
import { jsx as _jsx } from 'react/jsx-runtime';
const el = _jsx(Button, { kind: 'primary', children: 'Save' });

// ...which returns a plain, immutable description object
// (an "element"):
// {
//   // marks it as a real React element (security check)
//   $$typeof: Symbol(react.transitional.element),
//   // a component function, or a string like 'div'
//   type: Button,
//   // identity among siblings (2.4)
//   key: null,
//   props: { kind: 'primary', children: 'Save' }
// }
```

How it works: creating elements is cheap — it calls no component and touches no DOM. `Button` itself runs only when React decides to render it. That's why you can create elements freely and pass them around (as `children`, in arrays, in variables).

2.2 Fiber: the unit of work

Every element instance on screen gets a **fiber**: an internal object that remembers the component's type, key, props, its hooks' state (a linked list in `memoizedState`), pending updates, pointers to child/sibling/parent fibers, a priority (`lanes`), and flags describing DOM work to do. React keeps two fiber trees: `current` (what's on screen) and `workInProgress` (the next version), and swaps them when a render is committed.

```
setState(...)
  → an update is queued on the component's fiber,
    with a priority (lane)

render phase (interruptible)
  beginWork(fiber)
    → calls your component, gets new elements
  reconcileChildren
    → compares new elements with old fibers (2.3),
      marks changes
  ...may yield to the browser between fibers, then resume
  completeWork
    → prepares DOM nodes off-screen, bubbles flags up

commit phase (synchronous, never interrupted)
  mutation → insert/update/remove DOM nodes; refs
  layout   → useLayoutEffect runs (DOM updated,
             not yet painted)
  browser paints
  passive  → useEffect callbacks run
```

Hooks are stored **by call order** in that linked list: the first `useState` call reads slot 1, the second reads slot 2. That's the entire reason for the Rules of Hooks — calling a hook inside an `if` would shift every later hook to the wrong slot.

2.3 Reconciliation heuristics

Comparing two trees perfectly is O(n³). React's diff is O(n) because of two assumptions:

1. If the element `type` at a position changes, React destroys the old subtree (state is lost) and builds a new one.
2. Among siblings, `key` identifies which child is which across renders.

```
// State belongs to a POSITION + TYPE in the tree, not to a
// variable name.
function Form({ isLogin }: { isLogin: boolean }) {
  return isLogin ? (
    <Input placeholder="email" />
  ) : (
    <Input placeholder="username" />
  );
  // Both branches render an <Input> in the same position →
```

```
  // React treats it as the SAME component and keeps its
  // internal state (typed text survives switching modes).
  // Often surprising!
}

// Different keys tell React "these are different things" →
// the Input is recreated, state reset:
return isLogin ? <Input key="login" /> : <Input key="signup" />;

// Classic pattern: reset an entire form when the entity being
// edited changes. New user.id → fresh form, no stale state.
<EditUserForm key={user.id} user={user} />
```

```
// ❌ Defining a component inside another component
function Parent() {
  // a NEW function (= a new component type) on every render
  // of Parent
  const Child = () => <input />;
  // type changed → React unmounts and remounts Child every
  // time: the input loses focus and text after each keystroke
  return <Child />;
}
// ✅ Define components at module level (or pass data via
// props instead)
```

2.4 Keys done right

```
// ❌ Index as key: if you insert at the top or reorder, index 0
// now points to a different todo, but React reuses the old
// row's state and DOM (checkbox state, input text) for it →
// wrong UI.
{todos.map((t, i) => <TodoRow key={i} todo={t} />)}

// ✅ A stable, unique id from your data
{todos.map((t) => <TodoRow key={t.id} todo={t} />)}

// Index is acceptable ONLY for static lists that never reorder
// and whose rows hold no state. Never generate keys during
// render (Math.random(), crypto.randomUUID()): new key every
// render = every row remounts every time.
```

2.5 Batching and update queuing

Since React 18, multiple state updates in the same tick are **batched** into one re-render — in event handlers, promises, timeouts, everywhere.

```
function Counter() {
  const [n, setN] = useState(0);

  const add3Wrong = () => {
    // `n` is a constant captured by this render (a
    // snapshot), e.g. 0.
    setN(n + 1); // queue "set to 1"
    setN(n + 1); // queue "set to 1" again
    setN(n + 1); // queue "set to 1" again → result: 1
  };

  const add3Right = () => {
    // Updater functions receive the latest queued value, so
    // they chain correctly.
    setN((prev) => prev + 1); // 0 → 1
    setN((prev) => prev + 1); // 1 → 2
    setN((prev) => prev + 1); // 2 → 3, one re-render
  };

  const logAfterSet = () => {
    setN(n + 1);
    // still the OLD value — state updates apply on the next
    // render, not immediately
    console.log(n);
  };
}
```

`flushSync(() => setX(1))` forces React to apply an update synchronously before the next line (e.g. to scroll to a just-added list item). Use rarely — it defeats batching.

2.6 Bail-outs: when React skips work

- **Same state**: `setState(sameValue)` (compared with `Object.is`) → React bails out; the component may render once more but its children are skipped.
- **`memo(Component)`**: skips re-rendering when all props are shallowly equal to last time.
- **Children-as-props**: elements created by a *parent* don't re-render when a *child* component's state changes.

```
function ColorPicker({
  children,
}: {
  children: React.ReactNode;
}) {
  const [color, setColor] = useState('red');
  // When color changes, ColorPicker re-renders. But
  // `children` is the SAME element object that App created
  // earlier — React sees an identical element and skips
  // re-rendering it.
  return (
    <div style={{ color }}>
      <input onChange={(e) => setColor(e.target.value)} />
      {children}
    </div>
  );
}
function App() {
  // SlowTree is created by App, not ColorPicker
  return (
    <ColorPicker>
      <SlowTree />
    </ColorPicker>
  );
}
```

How it works: re-rendering means "call the component function again". The `<SlowTree />` element was produced by `App`'s last render; `ColorPicker` merely places it. Since `App` didn't re-render, the element is the same object and React skips it. This structural trick often beats `memo`.

2.7 StrictMode in development

StrictMode (dev only, zero cost in production) deliberately:

- calls component bodies, state initializers and reducers **twice** — impure code shows inconsistent results;
- mounts → unmounts → remounts every component once — effects without proper cleanup leak visibly (two subscriptions, two intervals);
- calls ref callbacks twice.

If your app breaks under StrictMode, the code is impure or lacks cleanup: fix the code, don't remove StrictMode.

2.8 What "render must be pure" means

```
// ❌ Mutating props during render: sort() sorts IN PLACE,
// changing the parent's array
function List({ items }: { items: Item[] }) {
  items.sort(byName);
  return <ul>{items.map(renderItem)}</ul>;
}

// ❌ Mutating a module variable during render: StrictMode's
// double render makes count jump by 2, and concurrent
// rendering may discard a render that already incremented it.
let count = 0;
function C() {
  count++;
  return <p>{count}</p>;
}

// ✅ Create new values instead of mutating
function List({ items }: { items: Item[] }) {
  // ES2023: returns a sorted COPY
  const sorted = items.toSorted(byName);
  return <ul>{sorted.map(renderItem)}</ul>;
}
```

Mutating objects you created *during this same render* is fine (building an array, then returning it). Reading or writing `ref.current` during render is not — refs are for event handlers and effects.

3. Components and TypeScript patterns

Type props as plain object types, extend native element props for wrapper components, and in React 19 pass `ref` as a normal prop — `forwardRef` is no longer needed.

3.1 Props basics

```
// ComponentPropsWithoutRef<'button'> = every prop a native
// <button> accepts (onClick, type, disabled, aria-*...).
// Intersecting (&) adds our own props on top.
type ButtonProps =
  React.ComponentPropsWithoutRef<'button'> & {
    // union of string literals = only these values allowed
    variant?: 'primary' | 'ghost' | 'danger';
    size?: 'sm' | 'md';
    loading?: boolean;
  };

export function Button({
  // default values via destructuring
  variant = 'primary',
  size = 'md',
  loading,
  children,
  disabled,
  // everything else (onClick, type, aria-label...) is
  // forwarded untouched
  ...rest
}: ButtonProps) {
  return (
    <button
      // spread first so our props below can't be
      // overridden by accident
      {...rest}
      // can't click while loading
      disabled={disabled || loading}
      // style hooks for CSS: [data-variant='danger'] {...}
      data-variant={variant}
      data-size={size}
    >
      {loading ? <Spinner /> : children}
    </button>
  );
}
```

- Don't use `React.FC` — it adds nothing useful and historically added an implicit `children` prop.

- `React.ReactNode` = anything renderable (strings, numbers, elements, arrays, null); use `React.ReactElement` only when you need an actual element.
- Event types: `React.ChangeEvent<HTMLInputElement>`, `React.FormEvent<HTMLFormElement>`, `React.MouseEvent<HTMLButtonElement>`.

3.2 ref as a prop (React 19)

```
// React 19: function components receive `ref` like any
// other prop. No forwardRef wrapper.
// ComponentProps<'input'> (WITH ref) includes the ref type.
function TextField({
  ref,
  label,
  ...rest
}: React.ComponentProps<'input'> & { label: string }) {
  // unique id linking <label> and <input> (accessibility)
  const id = useId();
  return (
    <>
      <label htmlFor={id}>{label}</label>
      {/* the parent's ref ends up on the real <input> */}
      <input id={id} ref={ref} {...rest} />
    </>
  );
}

// Usage: the parent can now focus the input or read it
const inputRef = useRef<HTMLInputElement>(null);
<TextField ref={inputRef} label="Email" />
// later, in an event handler: inputRef.current?.focus();
```

Ref callbacks can return a cleanup function (React 19) — perfect for observers attached to a DOM node:

```
<div
  ref={(node) => {
    // Called when the <div> is attached to the DOM
    const ro = new ResizeObserver(() => {
      /* react to size changes */
    });
    ro.observe(node!);
    // called when the <div> is removed — no leak
    return () => ro.disconnect();
  }}
/>
```

3.3 Exposing an imperative API

```
// Describe what the parent is allowed to call — NOT the raw
// <video> element.
type VideoHandle = { play(): void; pause(): void };

function Video({
  ref,
  src,
}: {
  ref?: React.Ref<VideoHandle>;
  src: string;
}) {
  // internal ref to the real DOM node
  const el = useRef<HTMLVideoElement>(null);
  // useImperativeHandle sets the parent's ref.current to the
  // object we return; [] = create the handle once.
  useImperativeHandle(
    ref,
    () => ({
      play: () => el.current?.play(),
      pause: () => el.current?.pause(),
    }),
    [],
  );
  return <video ref={el} src={src} />;
```

```
}
// Parent:
// const v = useRef<VideoHandle>(null);
// v.current?.play();
```

Prefer declarative props (`playing={true}`) when possible; use imperative handles only for focus, scroll, media playback and animations — things that are naturally commands.

3.4 Discriminated unions for props

```
// One shared literal field ("kind") lets TypeScript know
// which other props exist.
type AlertProps =
  | { kind: 'info'; message: string }
  | {
      kind: 'confirm';
      message: string;
      onConfirm: () => void;
      onCancel: () => void;
    };

function Alert(props: AlertProps) {
  if (props.kind === 'confirm') {
    // Inside this branch TypeScript has "narrowed" props:
    // onConfirm/onCancel definitely exist.
    return (
      <Dialog
        onOk={props.onConfirm}
        onCancel={props.onCancel}
      >
        {props.message}
      </Dialog>
    );
  }
  // here props.onConfirm would be a type error
  return <Banner>{props.message}</Banner>;
}
// <Alert kind="info" message="Saved" onConfirm={...} />
// → compile error: impossible states are unrepresentable
```

3.5 Generic components

```
// <T> makes the component work for any item type while
// keeping full type safety.
type ListProps<T> = {
  items: readonly T[];
  // caller decides what the key is
  getKey: (item: T) => string | number;
  // caller decides how each item looks
  renderItem: (item: T) => React.ReactNode;
  empty?: React.ReactNode;
};

export function List<T>({
  items,
  getKey,
  renderItem,
  empty = 'Nothing here',
}: ListProps<T>) {
  if (!items.length) return <p>{empty}</p>;
  return (
    <ul>
      {items.map((i) => (
        <li key={getKey(i)}>{renderItem(i)}</li>
      ))}
    </ul>
  );
}

// T is inferred from `items` → `u` is typed as User in
// both callbacks, with autocomplete.
<List
  items={users}
  getKey={(u) => u.id}
  renderItem={(u) => u.name}
/>
```

```
// Generic Select: the value type flows from options to
// onChange
type SelectProps<V extends string> = {
  value: V;
  options: readonly { value: V; label: string }[];
  onChange: (v: V) => void;
```

```
};
function Select<V extends string>({
  value,
  options,
  onChange,
}: SelectProps<V>) {
  return (
    // The DOM only gives us a string; the cast is safe
    // because options only contain V values.
    <select
      value={value}
      onChange={(e) => onChange(e.target.value as V)}
    >
      {options.map((o) => (
        <option key={o.value} value={o.value}>
          {o.label}
        </option>
      ))}
    </select>
  );
}
// <Select value={status} options={statusOptions}
//   onChange={setStatus} />
// setStatus receives 'open' | 'closed', not string
```

3.6 Polymorphic `as` prop

```
// A component that can render as different HTML elements
// while accepting THAT element's props.
type PolymorphicProps<
  E extends React.ElementType,
  P = object,
> = P & {
  as?: E;
  // native props of E, minus our own
} & Omit<React.ComponentPropsWithoutRef<E>, keyof P | 'as'>;

type TextOwnProps = { size?: 'sm' | 'md' | 'lg' };

function Text<E extends React.ElementType = 'span'>({
  as,
  size = 'md',
```

```
  ...rest
}: PolymorphicProps<E, TextOwnProps>) {
  // capitalized variable → JSX treats it as a component/tag
  const Comp = as ?? 'span';
  return <Comp data-size={size} {...rest} />;
}

// href allowed because E = 'a'
<Text as="a" href="/docs">Docs</Text>
// htmlFor allowed because E = 'label'
<Text as="label" htmlFor="x">Name</Text>
```

The modern alternative used by Radix and shadcn/ui is **`asChild`** (the Slot pattern): the component merges its props and behavior onto its single child element.

```
<Button asChild>
  {/* renders an <a> with Button's classes and handlers */}
  <Link to="/settings">Settings</Link>
</Button>
```

3.7 Slots via props

```
// Named slots are explicit and type-safe — better than
// inspecting children with React.Children.
type CardProps = {
  header: React.ReactNode;
  footer?: React.ReactNode;
  children: React.ReactNode;
};
function Card({ header, footer, children }: CardProps) {
  return (
    <section>
      <header>{header}</header>
      {children}
      {footer && <footer>{footer}</footer>}
    </section>
  );
}
```

`React.Children` and `cloneElement` still exist but are considered legacy; prefer explicit props, context, or render props.

3.8 Useful utility types

```
// props of an existing component (great for wrappers)
type P = React.ComponentProps<typeof Button>;
// native input props including ref
type InputProps = React.ComponentPropsWithRef<'input'>;
// type of a useState setter passed as a prop
type Setter<T> = React.Dispatch<React.SetStateAction<T>>;
// what a ref to <Video> points to (VideoHandle)
type El = React.ComponentRef<typeof Video>;

// `as const` keeps literal types: readonly ['sm','md']
const sizes = ['sm', 'md'] as const;
// 'sm' | 'md' — derive a type from runtime data
type Size = (typeof sizes)[number];

// Exhaustiveness check: if a new union member is added and
// not handled, this line fails to compile.
function assertNever(x: never): never {
  throw new Error(`Unhandled: ${String(x)}`);
}
```

3.9 Component design rules

- One component per responsibility; extract when a file passes ~200 lines or a component juggles more than 3 unrelated pieces of state.
- Containers fetch/own state; presentational components only receive props (easy to test and to show in Storybook).
- Reusable inputs should support both **controlled** (parent owns the value) and **uncontrolled** (component owns it) usage:

```
// If the parent passes `value`, it's controlled; otherwise
// we keep our own internal state.
function useControllable<T>({
  value,
  defaultValue,
  onChange,
}: {
  value?: T;
  defaultValue: T;
  onChange?: (v: T) => void;
}) {
```

```
  // used only in uncontrolled mode
  const [inner, setInner] = useState(defaultValue);
  const isControlled = value !== undefined;
  const current = isControlled ? value : inner;
  const set = (v: T) => {
    // uncontrolled: update our own state
    if (!isControlled) setInner(v);
    // always notify the parent
    onChange?.(v);
  };
  // tuple, like useState
  return [current, set] as const;
}
```

4. Hooks in depth

Every built-in hook, with the pitfall seniors are expected to know. Hooks are stored by call order on the fiber (ch. 2.2), so call them unconditionally at the top level of components or custom hooks — the new `use` API is the one exception.

4.1 useState

```
// explicit generic when the initial value can't tell the type
const [items, setItems] = useState<Item[]>([]);
// LAZY init: the function runs only on the first render
const [cfg] = useState(() => parseExpensiveConfig());
// useState(parseExpensiveConfig()) would CALL it on every
// render and throw the result away.

// Update immutably: create new arrays/objects so React sees
// a new reference.

// add
setItems((prev) => [...prev, newItem]);
// update one item
setItems((prev) =>
  prev.map((i) =>
    i.id === id ? { ...i, done: !i.done } : i,
  ),
);
// remove
```

```
setItems((prev) => prev.filter((i) => i.id !== id));
// nested: copy every level you change
setUser((u) => ({
  ...u,
  address: { ...u.address, city },
}));
// prev.push(x); setItems(prev) → same array reference →
// React may skip the re-render.
```

Common pitfalls: storing derived data (compute it during render instead), copying props into state (it won't follow later prop changes — reset with a `key` or derive), deeply nested state (normalize it or use Immer).

```
// ❌ Redundant state kept in sync by an effect: an extra
// render, and a frame where the two values disagree
const [fullName, setFullName] = useState('');
useEffect(
  () => setFullName(`${first} ${last}`),
  [first, last],
);

// ✅ Derive it — always correct, zero extra renders
const fullName = `${first} ${last}`;
```

Rare case — adjusting state when a prop changes, without an effect:

```
const [prevItems, setPrevItems] = useState(items);
// the prop changed since last render
if (items !== prevItems) {
  // remember the new one
  setPrevItems(items);
  // reset dependent state; React re-runs this render
  // immediately, before painting
  setSelection(null);
}
```

4.2 useReducer

Use when the next state depends on the previous state in several ways, or many events update related fields together.

```
// State as a discriminated union: each status carries
// exactly the fields that make sense for it.
type State = {
  status: 'idle' | 'loading' | 'success' | 'error';
  data?: User[];
  error?: string;
};
type Action =
  | { type: 'fetch' }
  | { type: 'resolve'; data: User[] }
  | { type: 'reject'; error: string };

// Pure function: (current state, action) → next state.
// No side effects, no mutation.
function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'fetch':
      return { status: 'loading' };
    case 'resolve':
      return { status: 'success', data: action.data };
    case 'reject':
      return { status: 'error', error: action.error };
    default:
      // compile error if a new action type is unhandled
      return assertNever(action);
  }
}

const [state, dispatch] = useReducer(reducer, {
  status: 'idle',
});
// dispatch({ type: 'resolve', data }) describes WHAT
// happened; the reducer decides how state changes
```

`dispatch` never changes identity, so it's safe to pass deep into the tree or list as an effect dependency. Reducers run twice in StrictMode — keep them pure.

4.3 useRef

A mutable box (`{ current }`) that survives re-renders and **does not** trigger a re-render when changed. Two uses: DOM handles and instance variables (timers, previous values, latest callbacks).

```
const timer = useRef<number | null>(null);
// store the id; no re-render needed
const start = () => {
  timer.current = window.setInterval(tick, 1000);
};
const stop = () => {
  if (timer.current) clearInterval(timer.current);
};

// Don't read or write ref.current during render (it makes
// rendering impure) — except lazy init:
const obs = useRef<IntersectionObserver | null>(null);
// create once, on the first render only
if (obs.current === null) {
  obs.current = new IntersectionObserver(cb);
}
```

4.4 useEffect / useLayoutEffect / useInsertionEffect

Hook	When it runs	Use for
`useEffect`	after the browser paints (doesn't block the screen)	subscriptions, network sync, analytics, non-React widgets
`useLayoutEffect`	after DOM changes, **before** paint (blocks paint)	measure layout then reposition (tooltips, popovers) without flicker
`useInsertionEffect`	before layout effects	CSS-in-JS libraries injecting `<style>` tags — not app code

```
function Tooltip({ anchor }: { anchor: DOMRect }) {
  const ref = useRef<HTMLDivElement>(null);
  const [h, setH] = useState(0);
  // Measure our own height after the DOM exists but before
  // the user sees anything. The setH re-render also happens
  // before paint → the user never sees the wrong position.
  useLayoutEffect(() => {
    setH(ref.current!.getBoundingClientRect().height);
  }, []);
  // Flip below the anchor if there isn't room above
```

```
  const top =
    anchor.top - h < 0 ? anchor.bottom : anchor.top - h;
  return (
    <div
      ref={ref}
      style={{ position: 'fixed', top, left: anchor.left }}
    >
      …
    </div>
  );
}
```

Effect discipline (when *not* to use them) is chapter 5.

4.5 useMemo / useCallback

```
// useMemo caches a computed VALUE until a dependency
// changes (compared with Object.is)
const visible = useMemo(
  () => filterTodos(todos, tab),
  [todos, tab],
);
// useCallback caches a FUNCTION identity — equivalent to
// useMemo(() => fn, deps)
const onSelect = useCallback(
  (id: string) => setSelected(id),
  [],
);
```

With the React Compiler enabled (ch. 6), you rarely write these by hand. Without it, use them only when: the computation is measurably expensive (>1 ms), the value is passed to a `memo` child, or it's a dependency of another hook. Memoizing cheap things costs more (memory + comparisons) than it saves.

4.6 useContext and the `use` API

```
// null default lets us detect "used outside the provider"
type Theme = 'light' | 'dark';
const ThemeContext = createContext<Theme | null>(null);

// React 19: the context object itself is the provider —
// <ThemeContext.Provider> is no longer needed
```

```
<ThemeContext value="dark">
  <App />
</ThemeContext>

// A custom hook gives every consumer one place for the
// error check and a non-null type
function useTheme() {
  // `use` may be called inside if/loops, unlike useContext
  const ctx = use(ThemeContext);
  if (!ctx) {
    throw new Error('useTheme must be used within ThemeContext');
  }
  // typed 'light' | 'dark'
  return ctx;
}
```

`use(promise)` suspends the component until the promise resolves (the nearest `<Suspense>` shows its fallback meanwhile). The promise must be **stable** — created outside render by a framework, a router loader, or a cache — never inline in the component, or every render creates a new promise and it suspends forever.

```
function Comments({
  commentsPromise,
}: {
  commentsPromise: Promise<Comment[]>;
}) {
  // throws to Suspense until resolved, then returns the value
  const comments = use(commentsPromise);
  return comments.map((c) => <p key={c.id}>{c.text}</p>);
}
```

4.7 useId

Generates an id that is stable across renders and identical on server and client (no hydration mismatch). For accessibility attributes — never for list keys.

```
// e.g. ":r3:" — unique per component instance
const id = useId();
<>
  <label htmlFor={`${id}-email`}>Email</label>
  <input
    id={`${id}-email`}
    aria-describedby={`${id}-hint`}
  />
  {/* screen readers read this after the label */}
  <p id={`${id}-hint`}>We never share it.</p>
</>
```

4.8 useSyncExternalStore

The correct way to read a mutable source outside React (browser APIs, your own store). It prevents **tearing**: during concurrent rendering, two components could otherwise read different values of the same store in one render.

```
function useOnlineStatus() {
  return useSyncExternalStore(
    // 1. subscribe: tell React how to listen for changes;
    //    return an unsubscribe function
    (cb) => {
      addEventListener('online', cb);
      addEventListener('offline', cb);
      return () => {
        removeEventListener('online', cb);
        removeEventListener('offline', cb);
      };
    },
    // 2. getSnapshot: the current value (must return the
    //    same value if nothing changed)
    () => navigator.onLine,
    // 3. getServerSnapshot: the value used during SSR
    () => true,
  );
}
```

```
// A minimal store compatible with useSyncExternalStore —
// essentially how Zustand works inside
function createStore<T>(initial: T) {
  let state = initial;
```

```
  const listeners = new Set<() => void>();
  return {
    get: () => state,
    set: (next: T | ((p: T) => T)) => {
      state =
        typeof next === 'function'
          ? (next as (p: T) => T)(state)
          : next;
      // notify subscribed components
      listeners.forEach((l) => l());
    },
    subscribe: (l: () => void) => {
      listeners.add(l);
      return () => listeners.delete(l);
    },
  };
}

// The selector picks a slice; React re-renders only if that
// slice changes (by Object.is). Selectors must return stable
// values (primitives, or the same object) — returning a
// fresh object every call would re-render forever.
function useStore<T, S>(
  store: ReturnType<typeof createStore<T>>,
  selector: (s: T) => S,
) {
  return useSyncExternalStore(store.subscribe, () =>
    selector(store.get()),
  );
}
```

4.9 useTransition and useDeferredValue

```
const [isPending, startTransition] = useTransition();
// Updates inside startTransition are low priority and
// interruptible: typing elsewhere stays instant.
const onTab = (t: Tab) => startTransition(() => setTab(t));

// React 19 "Actions": async functions inside transitions.
// isPending stays true until it finishes.
startTransition(async () => {
  const err = await updateName(name);
```

```
  // State updates after an `await` need their own
  // startTransition to stay part of the transition.
  startTransition(() => {
    if (err) setError(err);
    else navigate('/profile');
  });
});

// useDeferredValue returns a copy of a value that "lags
// behind" during heavy renders. Render the expensive list
// with it.
const deferredQuery = useDeferredValue(query);
// true while the list is catching up → dim it
const isStale = query !== deferredQuery;
```

Full explanation in chapter 7.

4.10 React 19 action hooks

```
// useActionState(action, initialState)
//   → [state, actionForTheForm, isPending]
const [state, formAction, isPending] = useActionState(
  // Receives the previous state and the submitted FormData;
  // returns the next state.
  async (prev: { error?: string }, formData: FormData) => {
    const res = await api.updateName(
      String(formData.get('name')),
    );
    // returned value becomes `state`
    return res.ok ? {} : { error: res.message };
  },
  // initial state
  {},
);

// React 19: functions are allowed in `action`
<form action={formAction}>
  <input name="name" />
  <SubmitButton />
  {state.error && <p role="alert">{state.error}</p>}
</form>

// useFormStatus reads the pending state of the PARENT
```

```
// <form> — no prop drilling. Great for design-system
// buttons used in many forms.
function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>Save</button>;
}

// useOptimistic shows the expected result immediately while
// the request runs.
const [optimisticTodos, addOptimistic] = useOptimistic(
  // real state from the server
  todos,
  // how to merge an optimistic item
  (current, t: Todo) => [...current, { ...t, sending: true }],
);
async function action(fd: FormData) {
  const todo = {
    id: crypto.randomUUID(),
    text: String(fd.get('text')),
  };
  // UI shows it right away (render it dimmed via `sending`)
  addOptimistic(todo);
  // When the action ends, optimistic state is dropped and
  // the real `todos` show. If createTodo throws, the
  // optimistic item disappears automatically.
  await createTodo(todo);
}
```

4.11 useEffectEvent (React 19.2)

Lets an effect read the **latest** props/state without listing them as dependencies — so changing them doesn't restart the effect.

```
function ChatRoom({
  roomId,
  theme,
}: {
  roomId: string;
  theme: Theme;
}) {
  // Always sees the current `theme`, but is NOT a reactive
  // dependency.
```

```
  const onConnected = useEffectEvent(() =>
    showToast('Connected', theme),
  );

  useEffect(() => {
    const conn = connect(roomId);
    conn.on('connected', onConnected);
    return () => conn.disconnect();
    // reconnects only when the room changes — not when the
    // theme changes
  }, [roomId]);
}
```

Only call effect events from inside effects; never pass them to children or call them during render.

4.12 Others worth knowing

- `useDebugValue(value)` — shows a label for a custom hook in React DevTools.
- `<Activity mode="hidden">` (19.2) — hide UI while keeping its state (ch. 7).
- `cache()` / `cacheSignal()` — Server Components only (Next.js guide).

4.13 Custom hooks

A custom hook shares **stateful logic**, not state — each component calling it gets its own independent state.

```
// Returns `value` only after it stopped changing for `ms`
// milliseconds (search inputs).
export function useDebounce<T>(value: T, ms = 300) {
  const [v, setV] = useState(value);
  useEffect(() => {
    // schedule the update
    const t = setTimeout(() => setV(value), ms);
    // a new keystroke cancels the previous timer
    return () => clearTimeout(t);
  }, [value, ms]);
  return v;
}

// State persisted to localStorage, surviving reloads.
export function useLocalStorage<T>(key: string, initial: T) {
```

```
  // lazy init: read storage once
  const [value, setValue] = useState<T>(() => {
    try {
      const raw = localStorage.getItem(key);
      return raw ? (JSON.parse(raw) as T) : initial;
    } catch {
      // corrupted JSON or storage disabled
      return initial;
    }
  });
  // write on change
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
  return [value, setValue] as const;
}

// Subscribes to a CSS media query (an external source →
// useSyncExternalStore).
export function useMediaQuery(q: string) {
  return useSyncExternalStore(
    (cb) => {
      const m = matchMedia(q);
      m.addEventListener('change', cb);
      return () => m.removeEventListener('change', cb);
    },
    () => matchMedia(q).matches,
    // server: assume no match
    () => false,
  );
}

// Window event listener that always calls the latest
// handler without re-subscribing.
export function useEventListener<
  K extends keyof WindowEventMap,
>(type: K, handler: (e: WindowEventMap[K]) => void) {
  const onEvent = useEffectEvent(handler);
  useEffect(() => {
    addEventListener(type, onEvent);
    return () => removeEventListener(type, onEvent);
  }, [type]);
}
```

Naming: always `useX` (the linter relies on it to enforce hook rules). Return a tuple for two values (`[value, set]`), an object for more. Keep hooks small and compose them.

5. Effects done right

An effect synchronizes a component with something **outside React**: the network, a DOM API, a timer, a WebSocket, a third-party widget. If no external system is involved, you almost certainly don't need an effect.

5.1 The decision tree

1. Can it be computed from props/state? → compute it during render (`useMemo` if expensive).
2. Is it a response to a user action? → put it in the event handler.
3. Is it fetching data? → use TanStack Query, a router loader or a framework — not a raw effect.
4. Is it subscribing to an external store? → `useSyncExternalStore`.
5. Must something stay in sync with an external system while the component is on screen? → `useEffect`.

5.2 Anti-patterns and fixes

```
// ❌ Event logic in an effect: it also fires on mount, on
// remount, and whenever `submitted` changes for unrelated
// reasons. The cause (a click) is far away from the effect.
useEffect(() => {
  if (submitted) post('/api/register', form);
}, [submitted]);

// ✅ Do it where the event happens
function handleSubmit() {
  post('/api/register', form);
}
```

```
// ❌ A chain of effects setting state: each link causes
// another render pass, and the logic is scattered across
// effects that trigger each other.
useEffect(() => {
  if (card.gold) setGoldCount((c) => c + 1);
```

```
}, [card]);
useEffect(() => {
  if (goldCount > 3) setRound((r) => r + 1);
}, [goldCount]);

// ✅ Compute all consequences in the handler that caused
// them — one render, readable logic
function placeCard(card: Card) {
  setCard(card);
  if (card.gold) {
    if (goldCount < 3) {
      setGoldCount(goldCount + 1);
    } else {
      // all these updates are batched into one render
      setGoldCount(0);
      setRound(round + 1);
    }
  }
}
```

```
// ❌ Notifying the parent from an effect: the parent
// updates one render late (visible flicker)
useEffect(() => {
  onChange(isOn);
}, [isOn, onChange]);

// ✅ Update both in the same event (or lift the state up so
// the parent owns it)
function toggle() {
  const next = !isOn;
  setIsOn(next);
  onChange(next);
}

// ❌ Resetting state when a prop changes: renders once with
// stale state, then again
useEffect(() => {
  setComment('');
}, [userId]);
```

```
// ✅ Give the component a key — React creates a fresh
// instance with fresh state
<Profile key={userId} userId={userId} />
```

5.3 Cleanup is mandatory for anything you start

```
useEffect(() => {
  // START syncing with the external system
  const conn = createConnection(serverUrl, roomId);
  conn.connect();
  // STOP: runs before the next effect run AND on unmount
  return () => conn.disconnect();
  // re-sync when either changes: disconnect old, connect new
}, [serverUrl, roomId]);

useEffect(() => {
  const id = setInterval(tick, 1000);
  // without this, every remount adds another interval
  return () => clearInterval(id);
}, []);

useEffect(() => {
  // a non-React widget owns this DOM node
  const map = new MapLibre({ container: ref.current! });
  // release its listeners, WebGL context, etc.
  return () => map.remove();
}, []);
```

Think in **start/stop pairs**, not "mount/unmount". StrictMode's development-only extra mount → unmount → mount proves your cleanup works: if something happens twice, cleanup is missing.

5.4 Race conditions when fetching manually

```
useEffect(() => {
  // lets us cancel this specific request
  const controller = new AbortController();
  setStatus('loading');
  fetch(`/api/users/${id}`, { signal: controller.signal })
    .then((r) => {
      // fetch only rejects on network errors, not 404/500
```

```
      if (!r.ok) throw new Error(r.statusText);
      return r.json();
    })
    .then((data) => {
      setUser(data);
      setStatus('success');
    })
    .catch((e) => {
      // an abort isn't a real error
      if (e.name !== 'AbortError') setStatus('error');
    });
  // If `id` changes (user clicks another profile) before the
  // response arrives, cleanup aborts the old request. Without
  // this, a slow response for the OLD id could arrive last
  // and overwrite the new user.
  return () => controller.abort();
}, [id]);
```

This is why data libraries exist: caching, deduplication, retries, background refetch, SSR and race handling are all hard to get right by hand. Use TanStack Query in real apps (ch. 9).

5.5 Dependencies: never lie to the linter

```
// ❌ An object created during render is a NEW object every
// render → Object.is says "changed" → the effect
// disconnects and reconnects on every render.
const options = { serverUrl, roomId };
useEffect(() => {
  const c = connect(options);
  return () => c.close();
}, [options]);

// ✅ Depend on primitives; build the object inside the effect
useEffect(() => {
  const c = connect({ serverUrl, roomId });
  return () => c.close();
}, [serverUrl, roomId]);
```

To remove a dependency, change the code so the effect doesn't need it — never silence the lint rule:

- use updater functions: `setCount((c) => c + 1)` instead of reading `count`;

- move a function used only by the effect *inside* the effect;
- move constants and pure helpers outside the component;
- use `useEffectEvent` for values the effect reads but shouldn't react to (ch. 4.11).

5.6 Code that should run once per app load

```
// Module-level flag: survives StrictMode's double mount and
// component remounts, because it lives outside any
// component instance.
let didInit = false;

function App() {
  useEffect(() => {
    if (didInit) return;
    didInit = true;
    loadDataFromLocalStorage();
    checkAuthToken();
  }, []);
}
// Even simpler: call these at module scope in main.tsx,
// before createRoot().render().
```

5.7 Ordering facts

- Children's effects run before their parents' effects (inside → out).
- All cleanups of a commit run before any new effect setups.
- `useLayoutEffect` cleanups and setups run before `useEffect` ones.
- An effect caused by a discrete event (a click) is flushed before the next discrete event is handled, so rapid clicks never see stale effects.

6. Performance

Measure first, then fix the biggest cost. In order of typical impact: bundle size and network waterfalls, then unnecessary re-renders of large subtrees, then expensive computations. The React Compiler removes most manual memoization work.

6.1 React Compiler

The compiler (stable since late 2025) analyzes components and hooks at build time and inserts fine-grained memoization — the equivalent of wrapping values, callbacks and JSX in `useMemo` / `useCallback` / `memo` exactly where it helps.

```
// What you write:
function TodoList({ todos, tab }: Props) {
  const visible = filterTodos(todos, tab);
  return (
    <List items={visible} onSelect={(id) => select(id)} />
  );
}

// What the compiler emits (simplified):
function TodoList(t0) {
  // an array of cache slots stored on this component's fiber
  const $ = _c(5);
  const { todos, tab } = t0;
  let visible;
  // inputs changed since last render?
  if ($[0] !== todos || $[1] !== tab) {
    // yes → recompute and remember
    visible = filterTodos(todos, tab);
    $[0] = todos;
    $[1] = tab;
    $[2] = visible;
  } else {
    // no → reuse the cached result (same reference)
    visible = $[2];
  }
  // ...the same treatment for the onSelect callback and the
  // <List> element, so <List> receives identical props and
  // React can skip re-rendering it.
}
```

Requirements: code must follow the Rules of React (pure render, no mutation of props/state, Rules of Hooks). Components that break the rules are skipped, and the ESLint plugin reports why. Incremental adoption:

```
// vite.config.ts: only compile files/functions that opt in
[
  'babel-plugin-react-compiler',
  { compilationMode: 'annotation' },
]
```

```
function Dashboard() {
  // opt this component in
  'use memo';
  /* ... */
}
function Legacy() {
  // opt out a component that breaks the rules until fixed
  'use no memo';
}
```

Keep a manual `useMemo` / `useCallback` only when it encodes meaning (e.g. a stable effect dependency you rely on); otherwise don't write them in new code.

6.2 Manual memoization (without the compiler)

```
// memo: re-render Row only if its props changed (shallow
// comparison of each prop)
const Row = memo(function Row({
  item,
  onSelect,
}: {
  item: Item;
  onSelect: (id: string) => void;
}) {
  return (
    <li onClick={() => onSelect(item.id)}>{item.name}</li>
  );
});

function Table({ items }: { items: Item[] }) {
  const [selected, setSelected] = useState<string>();
  // Without useCallback, onSelect would be a new function on
  // every Table render → every Row's props would "change" →
  // memo would be useless.
  const onSelect = useCallback(
    (id: string) => setSelected(id),
```

```
    [],
  );
  return (
    <ul>
      {items.map((i) => (
        <Row key={i.id} item={i} onSelect={onSelect} />
      ))}
    </ul>
  );
}
```

`memo` is useless if any prop is a new object, array or function every render. A custom comparator — `memo(C, (prev, next) => prev.id === next.id)` — is possible but easy to get wrong (stale props); use sparingly.

6.3 Structural fixes beat memoization

1. **Move state down** to the smallest component that needs it — fewer components re-render on change.
2. **Lift content up**: pass expensive subtrees as `children` (ch. 2.6).
3. **Split contexts** by update frequency; never put a fast-changing value in a context the whole app reads.
4. **Use selectors** with external stores (Zustand/Redux) so components subscribe only to their slice.

```
// Split state and dispatch into two contexts. Components
// that only dispatch (buttons) read CartDispatchCtx — whose
// value (dispatch) never changes — so they never re-render
// when the cart contents change.
const CartStateCtx = createContext<CartState | null>(null);
const CartDispatchCtx =
  createContext<React.Dispatch<CartAction> | null>(null);
```

6.4 Profiling workflow

1. React DevTools → Profiler → enable "Record why each component rendered" → reproduce the slow interaction → look for wide/tall bars in the flame graph and read the "why did this render" reason.
2. Chrome Performance panel: React 19.2 adds **Scheduler and Components tracks** showing render and commit work per priority alongside browser events.

3. DevTools "Highlight updates when components render" to spot unexpected re-render cascades visually.
4. `<Profiler id="Grid" onRender={(id, phase, actualDuration) => log(...)} />` to sample real render times in production.
5. Track Core Web Vitals from real users: **LCP** (largest content painted) < 2.5 s, **INP** (input responsiveness) < 200 ms, **CLS** (layout shift) < 0.1.

```
import { onLCP, onINP, onCLS } from 'web-vitals';

// Each callback fires with a metric object when the value
// is final; sendBeacon survives page unload.
const report = (m: unknown) =>
  navigator.sendBeacon('/vitals', JSON.stringify(m));

[onLCP, onINP, onCLS].forEach((fn) => fn(report));
```

6.5 Code splitting

```
// lazy() + dynamic import(): Vite puts Settings in its own
// JS file, downloaded only when rendered.
const Settings = lazy(() => import('./pages/Settings'));

// The fallback shows while the chunk downloads
<Suspense fallback={<PageSkeleton />}>
  <Settings />
</Suspense>

// Preload on intent: start downloading when the user hovers
// or focuses the link, so the chunk is often ready by the
// time they click. Same specifier → same cached module.
const loadSettings = () => import('./pages/Settings');
<Link
  to="/settings"
  onMouseEnter={loadSettings}
  onFocus={loadSettings}
>
  Settings
</Link>
```

Split by route first, then by heavy widgets (charts, rich-text editors, maps). Inspect the output with `npx vite-bundle-visualizer`. Import only what you use (`import { debounce } from 'es-toolkit'`, not an entire utility library).

6.6 Virtualization

Rendering 10,000 rows creates 10,000 DOM nodes. A virtualizer renders only the rows in (and slightly around) the viewport.

```
import { useVirtualizer } from '@tanstack/react-virtual';

function BigList({ rows }: { rows: Row[] }) {
  const parentRef = useRef<HTMLDivElement>(null);
  const v = useVirtualizer({
    count: rows.length,
    // the element that scrolls
    getScrollElement: () => parentRef.current,
    // row height guess in px (measured exactly if dynamic)
    estimateSize: () => 36,
    // extra rows above/below: no blank flashes when
    // scrolling fast
    overscan: 8,
  });
  return (
    <div
      ref={parentRef}
      style={{ height: 600, overflow: 'auto' }}
    >
      {/* A tall spacer so the scrollbar reflects the */}
      {/* full list height */}
      <div
        style={{
          height: v.getTotalSize(),
          position: 'relative',
        }}
      >
        {/* only the visible rows (~20), not 10,000 */}
        {v.getVirtualItems().map((vi) => (
          <div
            key={vi.key}
            style={{
              position: 'absolute',
```

```
              top: 0,
              transform: `translateY(${vi.start}px)`,
              height: vi.size,
              width: '100%',
            }}
          >
            {rows[vi.index]!.name}
          </div>
        ))}
      </div>
    </div>
  );
}
```

Rule of thumb: virtualize lists beyond ~200 complex rows.

6.7 Other wins

- **Images**: explicit `width` / `height` (prevents layout shift), `loading="lazy"` below the fold, AVIF/WebP, `srcset` for responsive sizes, `fetchpriority="high"` on the hero (LCP) image.
- **React 19 resource APIs**: `preload('/font.woff2', { as: 'font' })`, `preconnect('https://api.acme.com')`; `<title>`, `<meta>` and `<link>` rendered anywhere in the tree are hoisted into `<head>`.
- **High-frequency events** (search, resize, scroll): debounce/throttle the handler; for render-bound lag, prefer `useDeferredValue`.
- **Heavy CPU work**: move it to a Web Worker (Comlink makes calling worker functions feel like normal async calls).
- **Layout thrashing**: batch DOM reads before writes; animate only `transform` and `opacity` (they skip layout and paint).

7. Concurrent React: Suspense, transitions, error boundaries, Activity

Concurrent rendering lets React prepare a new UI in the background and interrupt that work for urgent input. You opt in per update with transitions; **Suspense** declares loading states; **error boundaries** declare failure states.

7.1 Update priorities (lanes)

Kind	Example	API
Urgent / discrete	typing, clicking, pressing keys	normal `setState` in event handlers
Transition	switching tabs, filtering a big list, navigation	`startTransition`, `useTransition`
Deferred	rendering a heavy view of a fast-changing value	`useDeferredValue`
Idle / offscreen	hidden tabs, pre-rendering	`<Activity mode="hidden">`

7.2 Transitions

```
function Tabs() {
  const [tab, setTab] = useState<'about' | 'posts'>('about');
  const [isPending, startTransition] = useTransition();
  // Marked as a transition: React may pause it to handle
  // more urgent input
  const showPosts = () =>
    startTransition(() => setTab('posts'));
  return (
    <>
      {/* Dim the nav while the new tab renders in the */}
      {/* background */}
      <nav style={{ opacity: isPending ? 0.6 : 1 }}>
        <button onClick={showPosts}>Posts</button>
      </nav>
      <Suspense fallback={<Spinner />}>
        {tab === 'posts' ? <Posts /> : <About />}
      </Suspense>
    </>
  );
}
```

How it works:

1. The click calls `setTab('posts')` inside a transition → React starts rendering `<Posts />` with low priority.

2. If `<Posts />` suspends (its data isn't loaded), React **keeps showing the old tab** instead of replacing it with the spinner — content the user already sees doesn't flash away.
3. `isPending` is true meanwhile, so you can show subtle feedback.
4. If the user types somewhere else, React pauses the transition render, handles the keystroke, then resumes.

You can't use transitions to update a controlled text input — input values must update urgently, or typing feels broken.

7.3 useDeferredValue for search

```
function Search() {
  const [query, setQuery] = useState('');
  // lags behind `query` while heavy rendering runs
  const deferred = useDeferredValue(query);
  const stale = query !== deferred;
  return (
    <>
      {/* The input uses the urgent value → no typing lag */}
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      {/* Results use the deferred value; dim them while */}
      {/* they're stale */}
      <div style={{ opacity: stale ? 0.5 : 1 }}>
        <Suspense fallback={<Spinner />}>
          <Results query={deferred} />
        </Suspense>
      </div>
    </>
  );
}

// memo is essential here (or the React Compiler): during
// the urgent render, Results receives the OLD deferred
// value — memo lets it skip that render entirely, so typing
// stays fast.
const Results = memo(function Results({
  query,
}: {
```

```
  query: string;
}) {
  /* heavy list */
});
```

Difference from debouncing: debounce waits a fixed time before doing anything; `useDeferredValue` starts rendering immediately and simply lets urgent updates interrupt it — faster on fast devices, graceful on slow ones.

7.4 Designing Suspense boundaries

```
{/* route level: shown until the page's essential */}
{/* data is ready */}
<Suspense fallback={<PageSkeleton />}>
  <Header />
  {/* independent region: the feed can load later */}
  <Suspense fallback={<FeedSkeleton />}>
    <Feed />
  </Suspense>
  {/* loads independently of the feed */}
  <Suspense fallback={<SidebarSkeleton />}>
    <Sidebar />
  </Suspense>
</Suspense>
```

- Things that suspend: `lazy()` components, `use(promise)`, Suspense-enabled data hooks (`useSuspenseQuery`), framework loaders / Server Components.
- When something suspends, React shows the **nearest** boundary's fallback above it.
- Place boundaries where a loading state makes sense to the user — not around every component.
- Skeletons should match the final layout so nothing jumps when content arrives (CLS).
- React 19 "pre-warms" siblings: when a boundary suspends, it shows the fallback immediately and then still renders siblings in the background so their data requests start early.

```
// TanStack Query with Suspense: the hook suspends until
// data exists, so `data` is never undefined.
function Feed() {
  const { data } = useSuspenseQuery({
    queryKey: ['feed'],
    queryFn: fetchFeed,
  });
  return data.map((p) => <Post key={p.id} post={p} />);
}
```

7.5 Error boundaries

Error boundaries catch errors thrown while **rendering** their children (and in lifecycle methods/constructors). They do **not** catch errors in event handlers, async code (`setTimeout`, promises), or inside the boundary itself. They're still class components under the hood; use `react-error-boundary`.

```
import { ErrorBoundary } from 'react-error-boundary';

function Fallback({
  error,
  resetErrorBoundary,
}: FallbackProps) {
  return (
    // role="alert" is announced by screen readers
    <div role="alert">
      <p>Something went wrong: {error.message}</p>
      {/* re-renders the children */}
      <button onClick={resetErrorBoundary}>Retry</button>
    </div>
  );
}

// QueryErrorResetBoundary resets failed TanStack queries
// when the boundary resets, so "Retry" actually refetches
// instead of re-throwing the cached error.
<QueryErrorResetBoundary>
  {({ reset }) => (
    <ErrorBoundary
      FallbackComponent={Fallback}
      onReset={reset}
      resetKeys={[userId]}
```

```
    >
      {/* resetKeys: navigating to another user also */}
      {/* clears the error */}
      <Suspense fallback={<Spinner />}>
        <UserProfile id={userId} />
      </Suspense>
    </ErrorBoundary>
  )}
</QueryErrorResetBoundary>
```

```
// Errors from event handlers/async code don't reach
// boundaries automatically — forward them:
const { showBoundary } = useErrorBoundary();
const onClick = async () => {
  try {
    await save();
  } catch (e) {
    // the nearest ErrorBoundary now renders its fallback
    showBoundary(e);
  }
};
```

Minimal class version, to understand what the library does:

```
type Props = {
  fallback: React.ReactNode;
  children: React.ReactNode;
};

class Boundary extends React.Component<
  Props,
  { hasError: boolean }
> {
  state = { hasError: false };
  // Called during render when a child throws → switch to
  // the fallback UI
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  // Called after commit with details → the place for logging
  componentDidCatch(error: Error, info: React.ErrorInfo) {
    reportError(error, info.componentStack);
  }
```

```
  render() {
    return this.state.hasError
      ? this.props.fallback
      : this.props.children;
  }
}
```

Layering: one boundary at the root (crash page), one per route, and one around risky widgets (charts, third-party embeds) so a broken widget doesn't take down the page.

7.6 Activity (React 19.2)

```
<Activity mode={tab === 'inbox' ? 'visible' : 'hidden'}>
  <Inbox />
</Activity>
// hidden: the DOM is kept but hidden (display: none),
// effects are cleaned up (subscriptions stop), state is
// preserved, and re-renders inside run at lowest priority.
// visible again: effects re-run and the UI reappears
// instantly, with scroll position and form input intact.
```

Use it for tab panels and back-navigation where users expect to return to exactly what they left, and to pre-render a likely next screen in the background. Compared with conditional rendering (`{tab === 'inbox' && <Inbox />}`) it keeps state; compared with CSS hiding it stops effects.

7.7 View Transitions (stabilizing)

React's `<ViewTransition>` component connects the browser's View Transitions API to transitions and Suspense reveals for animated UI changes. Treat it as experimental unless your React version documents it as stable; otherwise use `document.startViewTransition` directly or Motion (formerly Framer Motion).

8. State architecture

Classify state before choosing a tool. Most "global state" problems disappear once server data moves to TanStack Query and filters move to the URL; what remains (UI preferences, session, wizard progress) is small.

State type	Examples	Tool
Local UI	open/closed, hover, input draft	`useState` / `useReducer`
Shared UI (one subtree)	selected row in a table	lift state up, or Context
Global client	theme, auth session, cart, feature flags	Zustand / Redux Toolkit / Jotai
Server cache	users, projects — any API data	TanStack Query (ch. 9)
URL	filters, sort, page, selected tab, search	router search params
Form	field values, validation, dirty flags	React Hook Form / actions (ch. 11)
Persistent	drafts, preferences	localStorage via store middleware

8.1 Context: dependency injection, not a store

When a context's value changes, **every** component reading it re-renders. That's ideal for low-frequency values (theme, locale, current user, a service instance) and a poor fit for values changing many times per second.

```
type AuthCtx = {
  user: User | null;
  login: (c: Creds) => Promise<void>;
  logout: () => void;
};
const AuthContext = createContext<AuthCtx | null>(null);

export function AuthProvider({
  children,
}: {
  children: React.ReactNode;
}) {
  const [user, setUser] = useState<User | null>(null);
  const login = async (c: Creds) =>
    setUser(await api.login(c));
  const logout = () => {
```

```
    api.logout();
    setUser(null);
  };
  // Without memoization, `value` would be a new object every
  // render → all consumers re-render even if `user` didn't
  // change. (The React Compiler does this automatically.)
  const value = useMemo(
    () => ({ user, login, logout }),
    [user],
  );
  return <AuthContext value={value}>{children}</AuthContext>;
}

export const useAuth = () => {
  const ctx = use(AuthContext);
  // fail fast on wiring mistakes
  if (!ctx) throw new Error('useAuth outside AuthProvider');
  return ctx;
};
```

8.2 Zustand (default for global client state)

Zustand stores live outside React; components subscribe to **slices** with selector functions and re-render only when their slice changes.

```
import { create } from 'zustand';
import {
  persist,
  devtools,
  createJSONStorage,
} from 'zustand/middleware';
import { immer } from 'zustand/middleware/immer';

type CartItem = {
  id: string;
  name: string;
  price: number;
  qty: number;
};
type CartState = {
  items: CartItem[];
  // actions live in the store next to the data
```

```
  add: (item: Omit<CartItem, 'qty'>) => void;
  remove: (id: string) => void;
  clear: () => void;
};

export const useCart = create<CartState>()(
  // Middleware wraps from the inside out:
  // 3. inspect state/actions in Redux DevTools
  devtools(
    // 2. save to localStorage and rehydrate on load
    persist(
      // 1. actions "mutate" a draft; Immer produces a new
      //    immutable state
      immer((set) => ({
        items: [],
        add: (item) =>
          set((s) => {
            const found = s.items.find(
              (i) => i.id === item.id,
            );
            // looks like mutation — Immer makes it safe
            if (found) found.qty++;
            else s.items.push({ ...item, qty: 1 });
          }),
        remove: (id) =>
          set((s) => {
            s.items = s.items.filter((i) => i.id !== id);
          }),
        clear: () => set({ items: [] }),
      })),
      // storage key + backend
      {
        name: 'cart',
        storage: createJSONStorage(() => localStorage),
      },
    ),
  ),
);

// Derived selectors: components re-render only when the
// computed number changes.
export const useCartCount = () =>
  useCart((s) => s.items.reduce((n, i) => n + i.qty, 0));
```

```
export const useCartTotal = () =>
  useCart((s) =>
    s.items.reduce((t, i) => t + i.qty * i.price, 0),
  );
```

```
// Selecting several fields returns a new object each time →
// it would re-render on every store change. useShallow
// compares the object's fields one level deep instead.
import { useShallow } from 'zustand/react/shallow';
const { add, remove } = useCart(
  useShallow((s) => ({ add: s.add, remove: s.remove })),
);

// Outside React (API interceptors, tests): read and write
// the store directly
useCart.getState().clear();
```

Pitfall: `useCart()` with no selector subscribes to the entire store → re-renders on every change.

8.3 Redux Toolkit (large teams, strict conventions, complex event flows)

```
// features/todos/slice.ts
import {
  createSlice,
  createAsyncThunk,
  createEntityAdapter,
  type PayloadAction,
} from '@reduxjs/toolkit';

// Entity adapter: stores items normalized as
// { ids: [], entities: { [id]: item } } with CRUD helpers
const adapter = createEntityAdapter<Todo>();

// Async thunk: dispatches pending/fulfilled/rejected
// actions around the promise automatically
export const fetchTodos = createAsyncThunk(
  'todos/fetch',
  async () => (await api.get<Todo[]>('/todos')).data,
);

type Status = 'idle' | 'loading' | 'failed';
```

```
const todosSlice = createSlice({
  name: 'todos',
  initialState: adapter.getInitialState({
    status: 'idle' as Status,
  }),
  reducers: {
    // createSlice generates the action creator
    // `toggled(id)` and wires this reducer to it.
    toggled(state, action: PayloadAction<string>) {
      const t = state.entities[action.payload];
      // Immer inside → "mutation" is safe
      if (t) t.done = !t.done;
    },
  },
  extraReducers: (b) => {
    // Respond to the thunk's lifecycle actions
    b.addCase(fetchTodos.pending, (s) => {
      s.status = 'loading';
    })
      .addCase(fetchTodos.fulfilled, (s, a) => {
        s.status = 'idle';
        adapter.setAll(s, a.payload);
      })
      .addCase(fetchTodos.rejected, (s) => {
        s.status = 'failed';
      });
  },
});
export const { toggled } = todosSlice.actions;
// selectAll, selectById…
export const todosSelectors = adapter.getSelectors(
  (s: RootState) => s.todos,
);

// app/store.ts
export const store = configureStore({
  reducer: {
    todos: todosSlice.reducer,
    // RTK Query cache
    [api.reducerPath]: api.reducer,
  },
  middleware: (gdm) => gdm().concat(api.middleware),
```

```
});
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
// Pre-typed hooks: components never repeat the generics
export const useAppSelector =
  useSelector.withTypes<RootState>();
export const useAppDispatch =
  useDispatch.withTypes<AppDispatch>();
```

For server data inside a Redux app, use **RTK Query** (`createApi`) rather than hand-written thunks. Memoize derived data with `createSelector` .

8.4 Jotai (atomic state)

```
// a piece of state
const priceAtom = atom(10);
const qtyAtom = atom(2);
// Derived atom: recomputes only when priceAtom or qtyAtom
// change; only its readers re-render
const totalAtom = atom(
  (get) => get(priceAtom) * get(qtyAtom),
);
// Async atom: reading it suspends until resolved (works
// with <Suspense>)
const userAtom = atom(async (get) =>
  fetchUser(get(userIdAtom)),
);

function Total() {
  // subscribes to this atom only
  const total = useAtomValue(totalAtom);
  return <p>{total}</p>;
}
```

Good for editors and canvases with many independent, finely-updated pieces of state.

8.5 URL as state

```
import { useSearchParams } from 'react-router';

type StatusFilter = 'all' | 'open' | 'closed';
type Patch = Record<string, string | number | null>;
```

```
function useFilters() {
  // reads/writes ?status=open&page=2
  const [params, setParams] = useSearchParams();
  const status = (params.get('status') ??
    'all') as StatusFilter;
  // URL values are always strings → convert
  const page = Number(params.get('page') ?? 1);

  // Patch several params at once; null removes a param.
  // replace: don't add a history entry per keystroke.
  const set = (patch: Patch) =>
    setParams(
      (p) => {
        for (const [k, v] of Object.entries(patch)) {
          if (v === null) p.delete(k);
          else p.set(k, String(v));
        }
        return p;
      },
      { replace: true },
    );
  return { status, page, set };
}
```

URL state is shareable, bookmarkable, survives refresh, and works with the back button. Validate it with Zod (users can type anything) or use `nuqs` for typed search params.

8.6 State machines for complex flows

```
// Discriminated union: impossible combinations
// (uploading AND failed) can't be represented.
type Upload =
  | { s: 'idle' }
  | { s: 'uploading'; progress: number }
  | { s: 'failed'; error: string; retries: number }
  | { s: 'done'; url: string };
// Compare with
// { isLoading: boolean; error?: string; url?: string }
// — 8 combinations, most of them invalid.
```

For multi-step checkout, uploads with retry, or anything with many states and transitions, model it explicitly with a reducer over such a union, or with XState.

9. Server state with TanStack Query

Server state is a **cache of data you don't own**: it can go stale, it's shared by many components, and it needs deduplication, background refetching, retries and invalidation. TanStack Query v5 does all of that; your job is good query keys and correct invalidation.

9.1 Core concepts

Term	Meaning
`queryKey`	array that uniquely identifies cached data; everything the request depends on goes in the key
`staleTime`	how long data counts as fresh (no refetch). Default 0 = always considered stale
`gcTime`	how long unused data stays in memory after the last component stops using it. Default 5 min
`isPending`	no data yet (first load)
`isFetching`	any request in flight, including background refetches
`invalidateQueries`	mark matching queries stale and refetch the ones currently on screen

How a query behaves: component mounts → if the cache has data, it's returned immediately → if that data is stale, a background refetch runs and the component updates when it arrives. Two components using the same key share one request and one cache entry.

9.2 HTTP client and typed API layer

```
// shared/lib/http.ts — one fetch wrapper for the whole app
export class HttpError extends Error {
  constructor(
    public status: number,
    public body: unknown,
```

```
  ) {
    super(`HTTP ${status}`);
  }
}

export async function http<T>(
  path: string,
  init: RequestInit & { json?: unknown } = {},
): Promise<T> {
  const res = await fetch(`${env.VITE_API_URL}${path}`, {
    ...init,
    // send cookies (session auth)
    credentials: 'include',
    headers: {
      'Content-Type': 'application/json',
      ...init.headers,
    },
    body:
      init.json !== undefined
        ? JSON.stringify(init.json)
        : init.body,
  });
  // fetch resolves even for 404/500 — turn non-2xx into a
  // typed error so TanStack Query sees a failure
  if (!res.ok) {
    const body = await res.json().catch(() => null);
    throw new HttpError(res.status, body);
  }
  // 204 No Content has no body
  return res.status === 204
    ? (undefined as T)
    : res.json();
}
```

```
// features/projects/api.ts — keys + query definitions live
// together (single source of truth)
import { queryOptions } from '@tanstack/react-query';

// Validate responses at the boundary: if the backend
// changes shape, you get a clear error here instead of
// `undefined` crashes deep in components.
// z.coerce.date turns ISO strings into Date objects.
const ProjectSchema = z.object({
```

```
  id: z.string(),
  name: z.string(),
  updatedAt: z.coerce.date(),
});
export type Project = z.infer<typeof ProjectSchema>;
type Filters = { status?: string; q?: string };

// Key factory: hierarchical keys allow targeted or broad
// invalidation
export const projectKeys = {
  // ['projects']
  all: ['projects'] as const,
  // ['projects','list']
  lists: () => [...projectKeys.all, 'list'] as const,
  // ['projects','list',{status:'open'}]
  list: (f: Filters) =>
    [...projectKeys.lists(), f] as const,
  // ['projects','detail','42']
  detail: (id: string) =>
    [...projectKeys.all, 'detail', id] as const,
};

// queryOptions bundles key + fetcher + options, fully
// typed, reusable in hooks, loaders and prefetching
export const projectQueries = {
  list: (f: Filters) =>
    queryOptions({
      queryKey: projectKeys.list(f),
      // `signal` comes from TanStack Query: outdated
      // requests get aborted automatically
      queryFn: ({ signal }) => {
        const qs = new URLSearchParams(
          f as Record<string, string>,
        );
        return http<unknown>(`/projects?${qs}`, {
          signal,
        }).then((d) => z.array(ProjectSchema).parse(d));
      },
      staleTime: 60_000,
    }),
  detail: (id: string) =>
    queryOptions({
      queryKey: projectKeys.detail(id),
```

```
      queryFn: ({ signal }) =>
        http<unknown>(`/projects/${id}`, { signal }).then(
          (d) => ProjectSchema.parse(d),
        ),
    }),
};
```

9.3 Queries in components

```
function ProjectList() {
  // URL state (ch. 8.5)
  const { status, q } = useFilters();
  // don't fire a request per keystroke
  const debouncedQ = useDebounce(q, 300);
  const { data, isPending, isError, error, isFetching } =
    useQuery({
      // new filter → new key → new cache entry
      ...projectQueries.list({ status, q: debouncedQ }),
      // keep showing the previous results while the new
      // ones load
      placeholderData: keepPreviousData,
    });

  // nothing cached yet
  if (isPending) return <ListSkeleton />;
  if (isError) return <ErrorState error={error} />;
  return (
    <>
      {/* small indicator for background refreshes */}
      <Spinner hidden={!isFetching} />
      {data.map((p) => (
        <ProjectRow key={p.id} project={p} />
      ))}
    </>
  );
}

// Dependent query: wait until we know the user's org
const { data: user } = useQuery(userQueries.me());
const { data: org } = useQuery({
  ...orgQueries.detail(user?.orgId ?? ''),
  enabled: !!user?.orgId,
```

```
});

// A dynamic number of parallel queries
const results = useQueries({
  queries: ids.map((id) => projectQueries.detail(id)),
});

// select: transform or pick part of the data; the
// component re-renders only when the selected value changes
const { data: count } = useQuery({
  ...projectQueries.list({}),
  select: (d) => d.length,
});
```

9.4 Mutations, invalidation and optimistic updates

```
type Rename = { id: string; name: string };

export function useRenameProject() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ id, name }: Rename) =>
      http<Project>(`/projects/${id}`, {
        method: 'PATCH',
        json: { name },
      }),

    // 1. Before the request: update the cache optimistically
    onMutate: async ({ id, name }) => {
      const key = projectKeys.detail(id);
      // an in-flight refetch could overwrite our value
      await qc.cancelQueries({ queryKey: key });
      // snapshot for rollback
      const prev = qc.getQueryData(key);
      qc.setQueryData(key, (old: Project | undefined) =>
        old ? { ...old, name } : old,
      );
      // becomes `ctx` below
      return { prev };
    },
    // 2. On failure: restore the snapshot
    onError: (_err, { id }, ctx) =>
```

```
      qc.setQueryData(projectKeys.detail(id), ctx?.prev),
    // 3. Success or failure: refetch so the cache matches
    //    the server
    onSettled: (_data, _err, { id }) => {
      qc.invalidateQueries({
        queryKey: projectKeys.detail(id),
      });
      // prefix match: refreshes EVERY list variant
      qc.invalidateQueries({
        queryKey: projectKeys.lists(),
      });
    },
  });
}

// Usage
const rename = useRenameProject();
<button
  disabled={rename.isPending}
  onClick={() => rename.mutate({ id, name })}
>
  Save
</button>
```

Simpler optimistic UI in v5: skip the cache surgery and render `mutation.variables` (the pending input) while `mutation.isPending` is true.

9.5 Infinite queries

```
type Page = { items: Post[]; nextCursor: string | null };

const feed = useInfiniteQuery({
  queryKey: ['feed'],
  // pageParam is the cursor for the page being fetched
  queryFn: ({ pageParam, signal }) =>
    http<Page>(`/feed?cursor=${pageParam ?? ''}`, {
      signal,
    }),
  // the first page has no cursor
  initialPageParam: null as string | null,
  // return undefined/null → hasNextPage = false
  getNextPageParam: (last) => last.nextCursor,
```

```
});
// pages are kept separately; flatten for rendering
const posts =
  feed.data?.pages.flatMap((p) => p.items) ?? [];

// Load the next page when a sentinel element at the
// bottom scrolls into view
const sentinel = useRef<HTMLDivElement>(null);
useEffect(() => {
  const io = new IntersectionObserver(([entry]) => {
    const canLoad =
      feed.hasNextPage && !feed.isFetchingNextPage;
    if (entry?.isIntersecting && canLoad) {
      feed.fetchNextPage();
    }
  });
  if (sentinel.current) io.observe(sentinel.current);
  return () => io.disconnect();
}, [
  feed.hasNextPage,
  feed.isFetchingNextPage,
  feed.fetchNextPage,
]);
// <div ref={sentinel} /> placed after the list
```

9.6 Prefetching and router integration

```
// Prefetch on hover: by the time the user clicks, the data
// is often already cached
const qc = useQueryClient();
<Link
  to={`/projects/${p.id}`}
  onMouseEnter={() =>
    qc.prefetchQuery(projectQueries.detail(p.id))
  }
/>

// React Router loader: start fetching BEFORE the route
// component renders (no fetch-on-render waterfall)
export const projectLoader =
  (qc: QueryClient) =>
  async ({ params }: LoaderFunctionArgs) => {
```

```
    // returns cached data or fetches it
    await qc.ensureQueryData(
      projectQueries.detail(params.id!),
    );
    return null;
  };
// The route component then calls
// useSuspenseQuery(projectQueries.detail(id))
// and finds the data already cached.
```

9.7 Global error handling and session expiry

```
const isHttp = (e: unknown): e is HttpError =>
  e instanceof HttpError;

const queryClient = new QueryClient({
  queryCache: new QueryCache({
    // Runs once per failed query (not once per component
    // using it)
    onError: (error, query) => {
      if (isHttp(error) && error.status === 401) {
        // session expired
        useSession.getState().logout();
      } else if (query.state.data !== undefined) {
        // stale data still on screen → just notify
        toast.error(`Refresh failed: ${error.message}`);
      }
      // (with no data yet, the component shows its own
      // error state instead of a toast)
    },
  }),
  defaultOptions: {
    queries: {
      // Don't retry client errors (4xx — retrying won't
      // help); retry server errors up to 2 times
      retry: (count, err) =>
        !(isHttp(err) && err.status < 500) && count < 2,
    },
  },
});
```

9.8 Real-time updates

WebSocket/SSE events should update or invalidate the query cache rather than live in a separate store:

```
useEffect(() => {
  // server-sent events stream
  const es = new EventSource('/api/events');
  es.addEventListener('project.updated', (e) => {
    const p = ProjectSchema.parse(JSON.parse(e.data));
    // patch the detail cache directly
    qc.setQueryData(projectKeys.detail(p.id), p);
    // lists may need re-sorting → refetch
    qc.invalidateQueries({
      queryKey: projectKeys.lists(),
    });
  });
  return () => es.close();
}, [qc]);
```

10. Routing with React Router 7

React Router 7 (the merged Remix + React Router) has three modes: declarative (`<BrowserRouter>`), data mode (`createBrowserRouter` with loaders/actions — recommended for SPAs), and framework mode (Vite plugin, file routes, SSR — a Next.js alternative). Import everything from `react-router` .

10.1 Data router setup

```
// app/router.tsx
import {
  createBrowserRouter,
  RouterProvider,
  redirect,
} from 'react-router';

// The route tree is plain data: paths, components, and
// data functions (loader/action).
const router = createBrowserRouter([
  {
    path: '/',
```

```
    // renders on every page; children appear in <Outlet/>
    Component: RootLayout,
    // catches errors from any child loader/action/render
    ErrorBoundary: RootError,
    // shown while the initial loaders run
    HydrateFallback: FullPageSpinner,
    children: [
      // matches "/" exactly
      { index: true, Component: Home },
      // the action handles <Form method="post">
      {
        path: 'login',
        Component: Login,
        action: loginAction,
      },
      {
        // "pathless" layout route: groups routes without
        // adding a URL segment
        Component: ProtectedLayout,
        // runs BEFORE any child renders and redirects
        // anonymous users
        loader: requireAuth,
        children: [
          {
            path: 'projects',
            Component: Projects,
            loader: projectsLoader(queryClient),
          },
          {
            // :id becomes params.id
            path: 'projects/:id',
            // code-split: the module exports Component,
            // loader, action…
            lazy: () => import('@/routes/project-detail'),
          },
          {
            // * = nested routes defined in that module
            path: 'settings/*',
            lazy: () => import('@/routes/settings'),
          },
        ],
      },
      // anything unmatched
```

```
      { path: '*', Component: NotFound },
    ],
  },
]);

export function AppRouter() {
  return <RouterProvider router={router} />;
}
```

How it works: on navigation, the router matches the URL against the tree, runs **all matching loaders in parallel** (parent and child), waits for them, then renders the matched components. Data starts loading before components mount, which avoids "fetch in useEffect → render → fetch child data" waterfalls.

10.2 Layouts and outlets

```
function RootLayout() {
  // 'idle' | 'loading' (loaders running)
  //        | 'submitting' (action running)
  const navigation = useNavigation();
  return (
    <>
      <TopBar />
      {/* global navigation progress */}
      {navigation.state === 'loading' && <ProgressBar />}
      {/* the matched child route renders here */}
      <main>
        <Outlet />
      </main>
      {/* restores scroll position on back/forward */}
      <ScrollRestoration />
    </>
  );
}
```

10.3 Loaders, auth guards, params

```
// Used as the loader of ProtectedLayout: guards every
// child route at once
export async function requireAuth({
  request,
}: LoaderFunctionArgs) {
  // cached after the first check
  const session =
    await queryClient.ensureQueryData(sessionQuery);
  if (!session) {
    const from = new URL(request.url).pathname;
    // Throwing a redirect stops all loaders and navigates;
    // `from` lets login send the user back afterwards
    const to = `/login?from=${encodeURIComponent(from)}`;
    throw redirect(to);
  }
  return session;
}
```

```
// routes/project-detail.tsx — a lazy route module exports
// these named pieces
export async function loader({
  params,
}: LoaderFunctionArgs) {
  // from the :id segment
  const id = params.id!;
  // warm the TanStack Query cache
  await queryClient.ensureQueryData(
    projectQueries.detail(id),
  );
  // whatever is returned → useLoaderData()
  return { id };
}

export function Component() {
  // typed from the loader's return type
  const { id } = useLoaderData<typeof loader>();
  // already cached → no loading state
  const { data } = useSuspenseQuery(
    projectQueries.detail(id),
  );
  return <ProjectView project={data} />;
```

```
}

export function ErrorBoundary() {
  // whatever the loader/component threw
  const error = useRouteError();
  const notFound =
    isRouteErrorResponse(error) && error.status === 404;
  if (notFound) return <NotFound />;
  return <ErrorState error={error} />;
}
```

For expected errors, throw a response instead of an Error: `throw data('Not found', { status: 404 })` — the ErrorBoundary can then branch on the status.

10.4 Actions and `<Form>`

```
// An action handles a mutation for its route:
// <Form method="post"> submits here.
export async function loginAction({
  request,
}: ActionFunctionArgs) {
  // standard FormData
  const form = await request.formData();
  const parsed = LoginSchema.safeParse(
    Object.fromEntries(form),
  );
  if (!parsed.success) {
    // returned data → useActionData()
    return { errors: parsed.error.flatten().fieldErrors };
  }
  try {
    await api.login(parsed.data);
  } catch {
    return { errors: { form: ['Invalid credentials'] } };
  }
  const url = new URL(request.url);
  const from = url.searchParams.get('from') ?? '/projects';
  // success: navigate away
  return redirect(from);
}

function Login() {
```

```
  // undefined until the first submission
  const actionData = useActionData<typeof loginAction>();
  const nav = useNavigation();
  const errors = actionData?.errors;
  return (
    // <Form> from react-router: submits via fetch, no full
    // page reload
    <Form method="post">
      <input name="email" type="email" required />
      {errors?.email && <p>{errors.email[0]}</p>}
      <input name="password" type="password" required />
      <button disabled={nav.state === 'submitting'}>
        Sign in
      </button>
    </Form>
  );
}
```

After an action completes, React Router **re-runs the loaders** of the current page automatically, so the UI shows fresh data without manual cache updates. `useFetcher()` submits or loads without navigating — inline edits, "like" buttons, autosave:

```
const fetcher = useFetcher();
const busy = fetcher.state !== 'idle';
// Posts to the /projects/:id/star action; the page
// doesn't navigate
<fetcher.Form
  method="post"
  action={`/projects/${id}/star`}
>
  {/* pending feedback per button */}
  <button>{busy ? '…' : '★'}</button>
</fetcher.Form>
```

10.5 Navigation APIs

```
const navigate = useNavigate();
// replace: no back-button entry; state: hidden data
navigate('/projects', {
  replace: true,
  state: { flash: 'Saved' },
});
```

```
// go back
navigate(-1);

// NavLink knows whether its route is active (current) or
// pending (navigation to it in progress)
<NavLink
  to="/projects"
  className={({ isActive, isPending }) =>
    cx(isActive && 'font-bold', isPending && 'opacity-50')
  }
>
  Projects
</NavLink>

// Block navigation away from a dirty form (show your own
// confirm dialog when blocker.state === 'blocked')
const blocker = useBlocker(
  ({ currentLocation, nextLocation }) =>
    isDirty &&
    currentLocation.pathname !== nextLocation.pathname,
);
```

10.6 Type-safe alternative: TanStack Router

If you want fully typed paths, params and search params (validated by Zod) with file-based routes in a SPA, TanStack Router is the strongest option; its loaders integrate natively with TanStack Query. React Router's framework mode also generates route types (`+types/route`).

10.7 SPA deployment note

The server must serve `index.html` for every unknown path (history API fallback); otherwise refreshing `/projects/42` returns a 404. Configure it in your host: Firebase Hosting `rewrites` , Nginx `try_files $uri /index.html` , or a load-balancer rule.

11. Forms

Use React 19 actions (`<form action>` + `useActionState`) for simple forms and progressive enhancement; use React Hook Form + Zod for complex client forms (dynamic fields, cross-field validation, instant feedback). Share one Zod schema between client and server so validation rules exist once.

11.1 Controlled vs uncontrolled

- **Uncontrolled**: the DOM holds the value; you read it on submit via `FormData` or refs. Fewer re-renders, works with actions. The default for most forms.
- **Controlled**: React state holds the value (`value` + `onChange`). Use when the UI must react to every keystroke: formatting as you type, live previews, fields that depend on each other.

11.2 React 19 action form

```
const ContactSchema = z.object({
  email: z.string().email('Enter a valid email'),
  message: z.string().min(10, 'At least 10 characters'),
});
type Field = keyof z.infer<typeof ContactSchema> | 'form';

// What the action returns: success flag, per-field errors,
// and the submitted values (to refill the inputs)
type FormState = {
  ok?: boolean;
  errors?: Partial<Record<Field, string[]>>;
  values?: Record<string, string>;
};

async function sendContact(
  _prev: FormState,
  fd: FormData,
): Promise<FormState> {
  // { email: '...', message: '...' }
  const values = Object.fromEntries(fd) as Record<
    string,
    string
  >;
  // safeParse never throws
  const parsed = ContactSchema.safeParse(values);
  if (!parsed.success) {
    const errors = parsed.error.flatten().fieldErrors;
    return { errors, values };
  }
  try {
    await http('/contact', {
      method: 'POST',
```

```
      json: parsed.data,
    });
    return { ok: true };
  } catch {
    // form-level error
    const form = ['Could not send. Try again.'];
    return { errors: { form }, values };
  }
}

export function ContactForm() {
  const [state, action, pending] = useActionState(
    sendContact,
    {},
  );
  const err = state.errors;
  if (state.ok) return <p>Thanks!</p>;
  return (
    // noValidate: we show our own messages instead of the
    // browser's validation bubbles
    <form action={action} noValidate>
      {/* defaultValue (not value): uncontrolled input, */}
      {/* refilled from the last submission on error */}
      <input
        name="email"
        defaultValue={state.values?.email}
        aria-invalid={!!err?.email}
      />
      {err?.email && <p role="alert">{err.email[0]}</p>}
      <textarea
        name="message"
        defaultValue={state.values?.message}
      />
      {err?.message && (
        <p role="alert">{err.message[0]}</p>
      )}
      {err?.form && <p role="alert">{err.form[0]}</p>}
      <button disabled={pending}>
        {pending ? 'Sending…' : 'Send'}
      </button>
    </form>
  );
}
```

How it works: submitting calls `sendContact(previousState, formData)`; `pending` is true until it resolves; the returned object becomes `state` and the component re-renders. React resets uncontrolled fields after an action completes — that's why we return `values` on error and pass them back as `defaultValue`.

11.3 React Hook Form + Zod

```
import {
  useForm,
  useFieldArray,
  useWatch,
  Controller,
} from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';

const LineSchema = z.object({
  description: z.string().min(1),
  // <input type="number"> still yields strings → coerce
  qty: z.coerce.number().int().positive(),
  price: z.coerce.number().nonnegative(),
});

const InvoiceSchema = z
  .object({
    customer: z.string().min(1, 'Required'),
    // coerce: input strings → Date
    dueDate: z.coerce
      .date()
      .min(new Date(), 'Must be in the future'),
    lines: z.array(LineSchema).min(1, 'Add a line'),
    notes: z.string().max(500).optional(),
  })
  // cross-field rule on the whole object; `path` says
  // where the error is reported
  .refine(
    (v) =>
      v.lines.reduce((t, l) => t + l.qty * l.price, 0) > 0,
    { message: 'Total must be > 0', path: ['lines'] },
  );
// the TS type comes from the schema — no duplication
type Invoice = z.infer<typeof InvoiceSchema>;
const emptyLine = { description: '', qty: 1, price: 0 };
```

```
export function InvoiceForm({
  onSubmit,
}: {
  onSubmit: (v: Invoice) => Promise<void>;
}) {
  const {
    register,
    control,
    handleSubmit,
    setError,
    formState: { errors, isSubmitting, isDirty },
  } = useForm<Invoice>({
    // run Zod on validation
    resolver: zodResolver(InvoiceSchema),
    defaultValues: { customer: '', lines: [emptyLine] },
    // validate a field when it loses focus
    mode: 'onBlur',
  });

  // Dynamic list of line items; `fields` carry stable ids
  // for keys
  const { fields, append, remove } = useFieldArray({
    control,
    name: 'lines',
  });

  // only called with VALID, typed values
  const submit = handleSubmit(async (values) => {
    try {
      await onSubmit(values);
    } catch (e) {
      // Map server errors back onto fields
      if (e instanceof HttpError && e.status === 409) {
        setError('customer', { message: 'Duplicate' });
      } else {
        setError('root', { message: 'Server error' });
      }
    }
  });

  return (
    <form onSubmit={submit} noValidate>
```

```
      {/* register() wires name, ref, onChange, onBlur — */}
      {/* the input stays uncontrolled */}
      <Field
        label="Customer"
        error={errors.customer?.message}
      >
        <input {...register('customer')} />
      </Field>

      {/* Controller adapts components that don't expose */}
      {/* a native input (date pickers, selects) */}
      <Controller
        control={control}
        name="dueDate"
        render={({ field }) => (
          <DatePicker
            value={field.value}
            onChange={field.onChange}
          />
        )}
      />

      {fields.map((f, i) => (
        // f.id, never the index: removing a middle row
        // must not shift inputs
        <div key={f.id}>
          <input {...register(`lines.${i}.description`)} />
          <input
            type="number"
            {...register(`lines.${i}.qty`)}
          />
          <input
            type="number"
            step="0.01"
            {...register(`lines.${i}.price`)}
          />
          {/* type="button": don't submit the form */}
          <button type="button" onClick={() => remove(i)}>
            Remove
          </button>
        </div>
      ))}
      <button
```

```
        type="button"
        onClick={() => append(emptyLine)}
      >
        Add line
      </button>

      <InvoiceTotal control={control} />
      {errors.root && (
        <p role="alert">{errors.root.message}</p>
      )}
      <button disabled={isSubmitting || !isDirty}>
        Save
      </button>
    </form>
  );
}

// useWatch subscribes only THIS small component to line
// changes — the whole form doesn't re-render.
function InvoiceTotal({
  control,
}: {
  control: Control<Invoice>;
}) {
  const lines = useWatch({ control, name: 'lines' });
  const total = lines.reduce(
    (t, l) => t + Number(l.qty) * Number(l.price),
    0,
  );
  return <p>Total: {total.toFixed(2)}</p>;
}
```

Why React Hook Form is fast: inputs are uncontrolled and registered through refs, so typing doesn't re-render the form — only components subscribed to specific fields (`useWatch`, `formState` reads) re-render.

11.4 Reusable field component

```
// Wires label, error message and accessibility attributes
// around any single input child.
function Field({
  label,
```

```
  error,
  hint,
  children,
}: {
  label: string;
  error?: string;
  hint?: string;
  children: React.ReactElement<{ id?: string }>;
}) {
  const id = useId();
  // which message screen readers read after the label
  const describedBy = error
    ? `${id}-err`
    : hint
      ? `${id}-hint`
      : undefined;
  return (
    <div>
      <label htmlFor={id}>{label}</label>
      {cloneElement(children, {
        // links the label to the input
        id,
        // screen readers announce "invalid"
        'aria-invalid': !!error,
        'aria-describedby': describedBy,
      } as never)}
      {hint && !error && <p id={`${id}-hint`}>{hint}</p>}
      {error && (
        <p id={`${id}-err`} role="alert">
          {error}
        </p>
      )}
    </div>
  );
}
```

11.5 Form UX checklist

- Validate on blur, re-validate on change after the first error, and always validate again on the server — the client can be bypassed.
- Disable submit while pending; prevent double submission; show errors next to fields and move focus to the first invalid field.

- Use correct `type`, `autocomplete` (`email`, `current-password`, `one-time-code`) and `inputMode` attributes — password managers and mobile keyboards depend on them.
- Warn about unsaved changes with `useBlocker` plus a `beforeunload` listener.
- File uploads: check size and type on the client, upload directly to storage with presigned URLs, and show progress (`XMLHttpRequest.upload.onprogress` or a library).

12. Styling

The dominant 2026 stack: Tailwind CSS v4 for styling, CSS variables for design tokens, `cva` for component variants, and shadcn/ui (components copied into your repo, built on Radix or Base UI primitives) for accessible building blocks. Runtime CSS-in-JS (styled-components, Emotion) is declining: it costs runtime work and conflicts with Server Components.

12.1 Tailwind v4 setup (CSS-first config)

```
/* src/index.css */
/* loads Tailwind's base, components and utilities */
@import 'tailwindcss';

/* @theme defines design tokens. Each variable becomes
   utilities automatically:
   --color-brand-500 → bg-brand-500, text-brand-500 ... */
@theme {
  /* OKLCH: perceptually even lightness steps */
  --color-brand-50: oklch(0.97 0.02 260);
  --color-brand-500: oklch(0.6 0.2 260);
  --color-brand-600: oklch(0.52 0.21 260);
  /* → font-sans */
  --font-sans: 'Inter Variable', system-ui, sans-serif;
  /* → rounded-card */
  --radius-card: 0.75rem;
  /* → 3xl: responsive prefix */
  --breakpoint-3xl: 120rem;
}

/* Semantic tokens that switch with the theme: components
   use "fg"/"bg", never raw colors */
:root {
```

```
  --bg: white;
  --fg: oklch(0.2 0 0);
  --muted: oklch(0.55 0 0);
}
.dark {
  --bg: oklch(0.15 0 0);
  --fg: oklch(0.95 0 0);
  --muted: oklch(0.7 0 0);
}
/* inline: utilities reference the variable itself, so
   they update when .dark changes it */
@theme inline {
  --color-bg: var(--bg);
  --color-fg: var(--fg);
  --color-muted: var(--muted);
}

/* dark: variant activates under an element with class
   "dark" (instead of the OS media query) */
@custom-variant dark (&:where(.dark, .dark *));
```

No `tailwind.config.js` is needed in v4, and content detection (which files to scan for class names) is automatic.

12.2 Variants with cva + tailwind-merge

```
import {
  cva,
  type VariantProps,
} from 'class-variance-authority';
import { twMerge } from 'tailwind-merge';
import { clsx, type ClassValue } from 'clsx';

// clsx joins conditional classes; twMerge resolves
// conflicts ("px-2 px-4" → "px-4"), so a className passed
// by the caller reliably overrides the defaults.
export const cn = (...c: ClassValue[]) =>
  twMerge(clsx(c));

// base classes applied to every variant
const base = [
  'inline-flex items-center justify-center gap-2',
  'rounded-md font-medium transition-colors',
```

```
  'focus-visible:outline-2 focus-visible:outline-offset-2',
  'disabled:pointer-events-none disabled:opacity-50',
].join(' ');

const button = cva(base, {
  variants: {
    variant: {
      primary: 'bg-brand-600 text-white hover:bg-brand-500',
      // fg color at 5% opacity
      ghost: 'hover:bg-fg/5',
      danger: 'bg-red-600 text-white hover:bg-red-500',
    },
    size: {
      sm: 'h-8 px-3 text-sm',
      md: 'h-10 px-4',
      lg: 'h-12 px-6 text-lg',
    },
  },
  defaultVariants: { variant: 'primary', size: 'md' },
});

// VariantProps extracts
// { variant?: 'primary'|'ghost'|'danger';
//   size?: 'sm'|'md'|'lg' } from the cva config
type Props = React.ComponentProps<'button'> &
  VariantProps<typeof button>;

export function Button({
  className,
  variant,
  size,
  ...rest
}: Props) {
  return (
    <button
      className={cn(button({ variant, size }), className)}
      {...rest}
    />
  );
}
```

12.3 shadcn/ui workflow

```
# sets up the cn helper, CSS variables, paths
npx shadcn@latest init
# copies these components into src/shared/ui
npx shadcn@latest add button dialog dropdown-menu \
  form input table sonner
```

The components are **copied into** `src/shared/ui` — you own and can modify them; they're not a dependency you can't change. They wrap headless primitives that already handle focus trapping, keyboard navigation, ARIA attributes and portals.

```
<Dialog>
  {/* asChild: DialogTrigger passes its behavior to our */}
  {/* Button instead of rendering its own <button> */}
  <DialogTrigger asChild>
    <Button>Delete</Button>
  </DialogTrigger>
  {/* portal + focus trap + Escape to close, built in */}
  <DialogContent>
    {/* the title is read by screen readers */}
    <DialogHeader>
      <DialogTitle>Delete project?</DialogTitle>
    </DialogHeader>
    <DialogFooter>
      <Button variant="danger" onClick={onDelete}>
        Delete
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>
```

12.4 CSS Modules (zero-dependency alternative)

```
/* Card.module.css — class names are scoped automatically
   (compiled to e.g. Card_card__x7f2) */
.card {
  border-radius: var(--radius-card);
  padding: 1rem;
  /* makes this element a size container for
     @container queries */
  container-type: inline-size;
```

```
}
/* state via a data attribute */
.card[data-elevated='true'] {
  box-shadow: 0 4px 16px rgb(0 0 0 / 0.1);
}
/* Responds to the CARD's width, not the viewport — the
   component adapts wherever it's placed */
@container (min-width: 480px) {
  .body {
    display: grid;
    grid-template-columns: 1fr 2fr;
  }
}
```

```
// s.card is the generated, unique class name
import s from './Card.module.css';

<div className={s.card} data-elevated={elevated}>
  <div className={s.body}>…</div>
</div>
```

Use `data-*` attributes for state-driven styles — cleaner than juggling class names, and they work in Tailwind too: `data-[state=open]:rotate-180`.

12.5 Theme switching without a flash

```
<!-- index.html, inline in <head> BEFORE the app bundle:
     it runs before first paint, so a dark-mode user never
     sees a white flash while React loads -->
<script>
  const prefersDark = matchMedia(
    '(prefers-color-scheme: dark)',
  ).matches;
  const t =
    localStorage.theme ?? (prefersDark ? 'dark' : 'light');
  document.documentElement.classList.toggle(
    'dark',
    t === 'dark',
  );
</script>
```

12.6 Modern CSS a senior should use

- **Container queries** (`@container`) for component-level responsiveness.
- **`:has()`** for parent-state styling: `.field:has(input:invalid) { … }`.
- **Logical properties** (`margin-inline` , `padding-block`) so layouts flip correctly for right-to-left languages.
- **`clamp()`** for fluid type: `font-size: clamp(1rem, 0.9rem + 0.5vw, 1.25rem)` (min, preferred, max).
- **Cascade layers** (`@layer reset, components, utilities`) to control which styles win regardless of specificity.
- **`prefers-reduced-motion`** to disable non-essential animation for users who request it.

12.7 Animation

CSS transitions for simple state changes; **Motion** (`motion/react` , formerly Framer Motion) for layout animations, exit animations and gestures:

```
import { AnimatePresence, motion } from 'motion/react';

// AnimatePresence keeps a removed child mounted long
// enough to play its `exit` animation
<AnimatePresence>
  {open && (
    <motion.div
      // starting state when it mounts
      initial={{ opacity: 0, y: 8 }}
      // animates to this
      animate={{ opacity: 1, y: 0 }}
      // plays when `open` becomes false, before unmount
      exit={{ opacity: 0, y: 8 }}
      transition={{ duration: 0.15 }}
    >
      <Menu />
    </motion.div>
  )}
</AnimatePresence>
```

13. Advanced component patterns

Good component APIs come from composition: small parts that combine, shared state through context, and logic separated from markup (headless hooks). These patterns power Radix, React Aria, TanStack Table and every serious design system.

13.1 Composition over configuration

```
// ❌ Prop explosion: every new layout need adds another
// prop, and the combinations multiply
<Card
  title="x"
  subtitle="y"
  icon={<I />}
  actions={[...]}
  footerText="…"
  showDivider
/>

// ✅ Composable parts: callers arrange what they need;
// Card itself stays small
<Card>
  <Card.Header>
    <Icon />
    <h3>Title</h3>
  </Card.Header>
  <Card.Body>…</Card.Body>
  <Card.Footer>
    <Button>Open</Button>
  </Card.Footer>
</Card>
```

13.2 Compound components with context

The parent owns the state; the parts read it from context. Callers get a flexible, readable API (`<Tabs><Tabs.List>…`) and the parts stay in sync automatically.

```
type TabsCtx = {
  value: string;
  setValue: (v: string) => void;
  baseId: string;
};
```

```
const TabsContext = createContext<TabsCtx | null>(null);
const useTabs = () => {
  const c = use(TabsContext);
  // parts used outside the parent = a bug
  if (!c) throw new Error('Tabs.* must be inside <Tabs>');
  return c;
};

type Children = { children: React.ReactNode };

// Parent: holds the selected tab (controlled or
// uncontrolled via useControllable, ch. 3.9)
function Tabs({
  defaultValue,
  value,
  onValueChange,
  children,
}: Children & {
  defaultValue: string;
  value?: string;
  onValueChange?: (v: string) => void;
}) {
  const [current, setCurrent] = useControllable({
    value,
    defaultValue,
    onChange: onValueChange,
  });
  // prefix for aria ids linking tabs to panels
  const baseId = useId();
  const ctx = { value: current, setValue: setCurrent, baseId };
  return <TabsContext value={ctx}>{children}</TabsContext>;
}

// Arrow-key navigation, as required by the WAI-ARIA tabs
// pattern
function TabList({ children }: Children) {
  const onKeyDown = (
    e: React.KeyboardEvent<HTMLDivElement>,
  ) => {
    const tabs = [
      ...e.currentTarget.querySelectorAll<HTMLButtonElement>(
        '[role=tab]',
      ),
```

```
    ];
    const i = tabs.indexOf(
      document.activeElement as HTMLButtonElement,
    );
    const n = tabs.length;
    // wrap around at both ends
    let next = -1;
    if (e.key === 'ArrowRight') next = (i + 1) % n;
    if (e.key === 'ArrowLeft') next = (i - 1 + n) % n;
    // move focus and select
    if (next >= 0) {
      tabs[next]!.focus();
      tabs[next]!.click();
    }
  };
  return (
    <div role="tablist" onKeyDown={onKeyDown}>
      {children}
    </div>
  );
}

function Tab({ value, children }: Children & { value: string }) {
  const { value: cur, setValue, baseId } = useTabs();
  const selected = cur === value;
  return (
    <button
      role="tab"
      id={`${baseId}-tab-${value}`}
      aria-selected={selected}
      // which panel this tab shows
      aria-controls={`${baseId}-panel-${value}`}
      // only the selected tab is in the Tab order
      // ("roving tabindex")
      tabIndex={selected ? 0 : -1}
      onClick={() => setValue(value)}
    >
      {children}
    </button>
  );
}

function TabPanel({
```

```
  value,
  children,
}: Children & { value: string }) {
  const { value: cur, baseId } = useTabs();
  // only the active panel renders
  if (cur !== value) return null;
  return (
    <div
      role="tabpanel"
      id={`${baseId}-panel-${value}`}
      aria-labelledby={`${baseId}-tab-${value}`}
    >
      {children}
    </div>
  );
}

// Attach parts as static properties →
// <Tabs.List>, <Tabs.Tab>, <Tabs.Panel>
Tabs.List = TabList;
Tabs.Tab = Tab;
Tabs.Panel = TabPanel;
```

13.3 Headless hooks (logic without markup)

```
type BtnProps = React.ComponentProps<'button'>;
type DivProps = React.ComponentProps<'div'>;

// All behavior and accessibility wiring; zero opinions
// about markup or styles.
function useDisclosure(initial = false) {
  const [open, setOpen] = useState(initial);
  const id = useId();
  return {
    open,
    onOpen: () => setOpen(true),
    onClose: () => setOpen(false),
    toggle: () => setOpen((o) => !o),
    // "Prop getters" return the props an element needs,
    // merging in the caller's own props.
    getTriggerProps: (p: BtnProps = {}) => ({
      ...p,
```

```
      'aria-expanded': open,
      'aria-controls': id,
      onClick: (e: React.MouseEvent<HTMLButtonElement>) => {
        // the caller's handler runs first — composition,
        // not replacement
        p.onClick?.(e);
        setOpen((o) => !o);
      },
    }),
    getContentProps: (p: DivProps = {}) => ({
      ...p,
      id,
      hidden: !open,
    }),
  };
}

// Consumers render any markup they like:
const d = useDisclosure();
<>
  <button {...d.getTriggerProps()}>FAQ</button>
  <div {...d.getContentProps()}>Answer</div>
</>
```

The prop-getter pattern is the same API TanStack Table and Downshift expose: the library owns behavior, you own the look.

13.4 Render props (inversion of control)

```
// The parent decides WHERE things go; the caller decides
// WHAT they look like.
<DataTable
  rows={users}
  renderEmpty={() => <EmptyUsers />}
  renderRow={(u) => <UserRow user={u} />}
/>
```

Most former render-prop use cases became hooks; keep render props when the component controls placement (rows inside a virtualized table, items inside a menu).

13.5 Higher-order components

A legacy pattern (a function that takes a component and returns a wrapped one). Prefer hooks, but you'll still see it for cross-cutting wrappers:

```
function withFeatureFlag<P extends object>(
  flag: string,
  Component: React.ComponentType<P>,
) {
  return function Flagged(props: P) {
    // renders nothing when the flag is off
    return useFlag(flag) ? <Component {...props} /> : null;
  };
}
const NewCheckout = withFeatureFlag(
  'new-checkout',
  Checkout,
);
```

13.6 Portals

A portal renders children into a different DOM node (usually `document.body`) while keeping them in the same **React** tree — context still works and events still bubble to React ancestors.

```
function Modal({
  open,
  onClose,
  children,
}: {
  open: boolean;
  onClose: () => void;
  children: React.ReactNode;
}) {
  const ref = useRef<HTMLDialogElement>(null);
  // Sync React's `open` prop with the native dialog's
  // imperative API
  useEffect(() => {
    if (open) ref.current?.showModal();
    else ref.current?.close();
  }, [open]);
  return createPortal(
```

```
    <dialog
      ref={ref}
      // fires on Escape and on close()
      onClose={onClose}
      // a click on the backdrop area closes it
      onClick={(e) => e.target === ref.current && onClose()}
    >
      {children}
    </dialog>,
    // escape parents with overflow:hidden / z-index
    document.body,
  );
}
```

The native `<dialog>` with `showModal()` gives focus trapping, Escape handling, an inert background and the browser's top layer for free. The Popover API (the `popover` attribute) does the same for non-modal popups.

13.7 Defaults through context in a design system

```
// A group sets the size once; every button inside reads
// it — no prop drilling. 'md' is the default when no group
// is present.
const SizeContext = createContext<'sm' | 'md'>('md');

function ButtonGroup({
  size,
  children,
}: {
  size: 'sm' | 'md';
  children: React.ReactNode;
}) {
  return (
    <SizeContext value={size}>
      <div role="group">{children}</div>
    </SizeContext>
  );
}
function GroupButton(props: ButtonProps) {
```

```
  const size = use(SizeContext);
  // an explicit size prop still wins (spread last)
  return <Button size={size} {...props} />;
}
```

13.8 Optimistic, undoable delete

```
function useUndoableDelete() {
  const qc = useQueryClient();
  return (item: Item) => {
    // snapshot for undo
    const prev = qc.getQueryData<Item[]>(['items']);
    // hide it immediately
    qc.setQueryData<Item[]>(['items'], (l) =>
      l?.filter((i) => i.id !== item.id),
    );
    // Delay the real DELETE: if the user clicks Undo within
    // 5 s, nothing is ever sent
    const timer = setTimeout(
      () => http(`/items/${item.id}`, { method: 'DELETE' }),
      5000,
    );
    toast('Deleted', {
      action: {
        label: 'Undo',
        onClick: () => {
          clearTimeout(timer);
          qc.setQueryData(['items'], prev);
        },
      },
    });
  };
}
```

Undo is friendlier than confirm dialogs for reversible actions — users act fast and recover from mistakes.

14. Testing

Test behavior the user can observe, not implementation details. The "testing trophy": static checks at the base (TypeScript + ESLint), **many integration tests** (component + hooks + mocked network), some unit tests for pure logic, and a handful of end-to-end tests for critical journeys.

14.1 Setup

```
// src/test/setup.ts
// Runs before every test file (vite.config.ts →
// test.setupFiles).

// adds matchers like toBeInTheDocument()
import '@testing-library/jest-dom/vitest';
import { cleanup } from '@testing-library/react';
import { server } from './msw/server';

// onUnhandledRequest: 'error' → a request with no mock
// handler fails the test, so tests never hit a real API.
beforeAll(() =>
  server.listen({ onUnhandledRequest: 'error' }),
);
afterEach(() => {
  // remove per-test overrides (server.use) so tests don't
  // leak into each other
  server.resetHandlers();
  // unmount components rendered in the test
  cleanup();
});
afterAll(() => server.close());
```

```
// src/test/render.tsx — render with every provider the app
// uses, so tests match production wiring
export function renderWithProviders(
  ui: React.ReactElement,
  { route = '/' } = {},
) {
  // A fresh QueryClient per test: no cache shared between
  // tests. retry: false → failed requests fail immediately
  // (tests stay fast and deterministic).
  const qc = new QueryClient({
```

```
    defaultOptions: {
      queries: { retry: false, gcTime: 0 },
    },
  });
  // Memory router: routing works without a real browser
  // URL; initialEntries sets the start page.
  const router = createMemoryRouter(
    [{ path: '*', element: ui }],
    { initialEntries: [route] },
  );
  return {
    // simulates real user interactions (focus, keydown,
    // input, click...)
    user: userEvent.setup(),
    qc,
    ...render(
      <QueryClientProvider client={qc}>
        <RouterProvider router={router} />
      </QueryClientProvider>,
    ),
  };
}
```

14.2 MSW: mock the network, not your modules

Mock Service Worker intercepts real `fetch` calls at the network level. Your components, hooks, http client and TanStack Query all run for real — only the server is fake.

```
// src/test/msw/handlers.ts
import { http, HttpResponse, delay } from 'msw';
import { setupServer } from 'msw/node';

export const handlers = [
  // '*/projects' matches any origin — works no matter what
  // VITE_API_URL is in tests
  http.get('*/projects', ({ request }) => {
    const url = new URL(request.url);
    const q = url.searchParams.get('q') ?? '';
    // in-memory fake data
    return HttpResponse.json(
      db.projects.filter((p) => p.name.includes(q)),
    );
```

```
  }),
  http.patch(
    '*/projects/:id',
    async ({ params, request }) => {
      const body = (await request.json()) as {
        name: string;
      };
      // realistic latency → lets you see pending states
      await delay(50);
      return HttpResponse.json({
        ...db.find(params.id as string),
        ...body,
      });
    },
  ),
];
export const server = setupServer(...handlers);
```

The same handlers can run in the browser (`setupWorker` from `msw/browser`) for local development without a backend, and in Storybook.

14.3 Component integration test

```
test('renames a project optimistically', async () => {
  const { user } = renderWithProviders(
    <ProjectPage id="p1" />,
  );

  // findBy* waits (retries) until the element appears —
  // the data arrives asynchronously
  expect(
    await screen.findByRole('heading', { name: 'Apollo' }),
  ).toBeInTheDocument();

  // regex: case-insensitive
  await user.click(
    screen.getByRole('button', { name: /rename/i }),
  );
  // found by its accessible label
  const input = screen.getByRole('textbox', {
    name: /name/i,
  });
  await user.clear(input);
```

```
  // {Enter} presses the Enter key
  await user.type(input, 'Artemis{Enter}');

  // Optimistic update: the new name shows before the 50 ms
  // mocked response returns
  expect(
    screen.getByRole('heading', { name: 'Artemis' }),
  ).toBeInTheDocument();
});

test('shows a server error and rolls back', async () => {
  // Override one handler for this test only (reset in
  // afterEach)
  server.use(
    http.patch('*/projects/:id', () =>
      HttpResponse.json(
        { message: 'nope' },
        { status: 500 },
      ),
    ),
  );
  const { user } = renderWithProviders(
    <ProjectPage id="p1" />,
  );
  // ...perform the rename as above...
  expect(await screen.findByRole('alert')).toHaveTextContent(
    /could not rename/i,
  );
  // rolled back
  expect(
    screen.getByRole('heading', { name: 'Apollo' }),
  ).toBeInTheDocument();
});
```

Query	Behavior	Use
`getBy*`	returns the element or throws immediately	it must exist right now
`queryBy*`	returns the element or `null`	asserting something is absent
`findBy*`	returns a promise, retries until found or timeout	something that appears asynchronously

Priority of selectors: `*ByRole` (with `name`) → `*ByLabelText` → `*ByText` → `*ByTestId` as a last resort. If you can't find an element by role, real users of assistive technology probably can't either.

14.4 Testing hooks

```
import { renderHook, act } from '@testing-library/react';

test('useDebounce delays updates', () => {
  // take control of setTimeout
  vi.useFakeTimers();
  // renderHook mounts a tiny test component that calls the
  // hook; rerender passes new props
  const { result, rerender } = renderHook(
    ({ v }) => useDebounce(v, 300),
    { initialProps: { v: 'a' } },
  );
  rerender({ v: 'ab' });
  // not updated yet
  expect(result.current).toBe('a');
  // act() flushes the resulting React update
  act(() => vi.advanceTimersByTime(300));
  expect(result.current).toBe('ab');
  vi.useRealTimers();
});
```

14.5 Unit tests for pure logic

```
describe('cart reducer', () => {
  // it.each runs the same test with different inputs:
  // [initial state, action, expected qty]
  it.each([
    [{ items: [] }, { type: 'add', item: apple }, 1],
    [
      { items: [{ ...apple, qty: 1 }] },
      { type: 'add', item: apple },
      2,
    ],
  ])('adds items', (state, action, expectedQty) => {
    const next = cartReducer(state, action);
    expect(next.items[0]!.qty).toBe(expectedQty);
  });
});
```

14.6 End-to-end with Playwright

```
// e2e/checkout.spec.ts — drives a real browser against the
// built app
import { test, expect } from '@playwright/test';

// Reuse a logged-in session saved once by a setup project
// (faster than logging in per test)
test.use({ storageState: 'e2e/.auth/user.json' });

test('user can check out', async ({ page }) => {
  await page.goto('/shop');
  await page
    .getByRole('button', { name: 'Add Apple to cart' })
    .click();
  await page
    .getByRole('link', { name: /cart \(1\)/i })
    .click();
  await page
    .getByRole('button', { name: 'Checkout' })
    .click();
  // test card number
  await page
    .getByLabel('Card number')
```

```
    .fill('4242 4242 4242 4242');
  await page.getByRole('button', { name: 'Pay' }).click();
  // expect(...).toBeVisible() auto-waits up to the
  // timeout — no manual sleeps
  await expect(
    page.getByRole('heading', { name: 'Order confirmed' }),
  ).toBeVisible();
});
```

```
// playwright.config.ts (essentials)
export default defineConfig({
  // Start the production preview server before tests;
  // reuse a running one locally
  webServer: {
    command: 'npm run preview',
    port: 4173,
    reuseExistingServer: !process.env.CI,
  },
  use: {
    baseURL: 'http://localhost:4173',
    // trace = replayable recording of failures
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium', use: devices['Desktop Chrome'] },
    // Safari engine + mobile viewport
    { name: 'webkit', use: devices['iPhone 15'] },
  ],
  // tolerate rare flakiness in CI only
  retries: process.env.CI ? 2 : 0,
});
```

14.7 Other layers

- **Storybook** (with interaction tests and the accessibility addon) documents and tests components in isolation; Chromatic or Playwright screenshots provide visual regression testing.
- **Accessibility in tests**: `vitest-axe` or `@axe-core/playwright` fail the build on detectable violations.
- **Coverage**: `vitest --coverage` (v8 provider). Aim for meaningful coverage of critical paths, not a percentage target.

- **Vitest Browser Mode** runs component tests in a real browser (via Playwright) when jsdom's gaps (layout, real CSS) matter.

15. Accessibility, internationalization, security

These are advanced-level responsibilities: accessibility is a legal requirement in many markets (the EU Accessibility Act has applied since June 2025; the ADA in the US), i18n is far cheaper designed in than retrofitted, and XSS and token-handling mistakes are the most common frontend security incidents.

15.1 Accessibility (WCAG 2.2 AA)

1. **Semantic HTML first**: `<button>` for actions, `<a href>` for navigation, landmarks (`<nav>`, `<main>`), headings in order, a `<label>` for every input. Semantics give keyboard support and screen-reader meaning for free.
2. **Keyboard**: every interactive element reachable with Tab, operable with Enter/Space, with a visible `:focus-visible` ring, and no keyboard traps.
3. **Focus management** on route changes and in dialogs:

```
// After client-side navigation, screen readers don't know
// the page changed (no full reload). Moving focus to the
// new page's heading makes them announce it.
function PageHeading({
  children,
}: {
  children: React.ReactNode;
}) {
  const ref = useRef<HTMLHeadingElement>(null);
  const location = useLocation();
  // re-run on every navigation
  useEffect(() => {
    ref.current?.focus();
  }, [location.pathname]);
  // tabIndex={-1}: focusable by script, but not added to
  // the Tab order
  return (
    <h1 ref={ref} tabIndex={-1}>
```

```
      {children}
    </h1>
  );
}
```

4. **Live regions** announce async changes that don't move focus:

```
// aria-live="polite": screen readers read changes to this
// text when they finish their current sentence.
// sr-only: visually hidden but still read by assistive
// technology.
<div aria-live="polite" className="sr-only">
  {isFetching ? 'Loading results' : `${count} results`}
</div>
```

5. **ARIA only when HTML can't express it** — "no ARIA is better than bad ARIA". Icon-only buttons need an `aria-label`.
6. **Contrast**: 4.5:1 for text, 3:1 for UI components; never use color as the only signal (add icons or text).
7. Respect `prefers-reduced-motion`; make touch targets at least 24×24 px.
8. **Tooling**: eslint-plugin-jsx-a11y, axe DevTools, Lighthouse — plus real screen-reader testing (VoiceOver, NVDA), which tools can't replace.

15.2 Internationalization

```
// i18next + react-i18next (most common; alternatives:
// react-intl, Lingui)
i18n
  .use(initReactI18next)
  .use(HttpBackend)
  .init({
    // used when a key is missing in the current language
    fallbackLng: 'en',
    supportedLngs: ['en', 'ka', 'de'],
    // namespaces: split translations per feature, load
    // them on demand
    ns: ['common', 'projects'],
    // React already escapes output; escaping twice would
    // show &amp;
    interpolation: { escapeValue: false },
  });
```

```
function Header({ count }: { count: number }) {
  // re-renders when the language changes
  const { t, i18n } = useTranslation('projects');
  return (
    <>
      <h1>{t('title')}</h1>
      {/* Plurals: i18next picks count_one / count_other */}
      {/* (and extra forms some languages need), e.g. */}
      {/* "count_one": "{{count}} project" */}
      {/* "count_other": "{{count}} projects" */}
      <p>{t('count', { count })}</p>
      <select
        value={i18n.language}
        onChange={(e) => i18n.changeLanguage(e.target.value)}
      >
        …
      </select>
    </>
  );
}
```

```
// Format with the built-in Intl APIs — never hand-format
// dates, numbers or currency.

// "1.234,50 €" in German
new Intl.NumberFormat(locale, {
  style: 'currency',
  currency: 'EUR',
}).format(1234.5);

// "21 Oct 2026"
new Intl.DateTimeFormat(locale, {
  dateStyle: 'medium',
}).format(date);

// "yesterday"
new Intl.RelativeTimeFormat(locale, {
  numeric: 'auto',
```

```
}).format(-1, 'day');

// "a, b, and c"
new Intl.ListFormat(locale).format(['a', 'b', 'c']);
```

Set `<html lang dir>`; use logical CSS properties for right-to-left languages; never concatenate translated fragments (word order differs per language — use interpolation instead); leave 30–40% extra room for text expansion.

15.3 Security

XSS (cross-site scripting). React escapes interpolated strings, so `{userInput}` is safe. The holes you must guard yourself:

```
// 1. dangerouslySetInnerHTML inserts raw HTML.
//    Always sanitize first.
import DOMPurify from 'dompurify';
// sanitize strips <script>, onerror=, etc.
<div
  dangerouslySetInnerHTML={{
    __html: DOMPurify.sanitize(userHtml),
  }}
/>

// 2. User-controlled URLs: "javascript:alert(1)" in an
//    href executes code when clicked. React 19 blocks
//    javascript: URLs, but validate anyway (defense in
//    depth).
const SAFE = ['http:', 'https:', 'mailto:'];
const safeUrl = (u: string) => {
  try {
    const { protocol } = new URL(u, location.href);
    return SAFE.includes(protocol) ? u : '#';
  } catch {
    // unparsable URL
    return '#';
  }
};
<a href={safeUrl(user.website)}>site</a>
```

```
// 3. Spreading untrusted objects as props
//    ({...userControlledProps}) could inject
//    dangerouslySetInnerHTML or event handlers.
//    Pick known fields explicitly.
```

Tokens. Prefer HttpOnly, Secure, SameSite cookies set by the backend: JavaScript can't read them, so an XSS bug can't steal them. If you must hold an access token in JS, keep it **in memory only** (not localStorage), short-lived, refreshed through an HttpOnly refresh cookie:

```
// module variable: gone on reload, unreachable from other
// origins
let accessToken: string | null = null;

async function authFetch(
  input: string,
  init: RequestInit = {},
) {
  const go = () =>
    fetch(input, {
      ...init,
      headers: {
        ...init.headers,
        Authorization: `Bearer ${accessToken}`,
      },
    });
  let res = await go();
  // access token expired
  if (res.status === 401) {
    // The refresh token lives in an HttpOnly cookie;
    // credentials: 'include' sends it
    const r = await fetch('/auth/refresh', {
      method: 'POST',
      credentials: 'include',
    });
    // refresh failed → the user must log in again
    if (!r.ok) throw new Error('Session expired');
    accessToken = (await r.json()).accessToken;
    // retry the original request once
```

```
    res = await go();
  }
  return res;
}
```

CSRF. With cookie auth, a malicious site can make the browser send requests with your cookies. Use `SameSite=Lax` / `Strict` cookies plus a CSRF token header (or an Origin check on the server) for state-changing requests.

CSP. Send a strict Content-Security-Policy header from your host so injected scripts can't run: `default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'`.

Secrets. Everything in the client bundle is public — including every `VITE_*` variable. API keys that cost money or grant access belong on a server.

Dependencies. Run `npm audit`, use Dependabot/Renovate, commit the lockfile, avoid abandoned packages, and review packages with install scripts — npm supply-chain attacks rose sharply in 2025.

Authorization. Hiding a button is UX, not security. Every permission check must also happen on the server.

16. Build, deploy, observability

A Vite SPA builds to static files: content-hashed JS/CSS cached forever, `index.html` never cached, all served from a CDN with an SPA fallback. Production readiness adds error tracking, performance monitoring and safe rollouts (feature flags). Deeper Google Cloud coverage is in the DevOps guide.

16.1 Build output and caching

```
dist/
  index.html
    → Cache-Control: no-cache
      (always revalidated: it points to the latest
       hashed files)
  assets/index-3f9a1c.js
    → Cache-Control: public, max-age=31536000,
      immutable
  assets/Settings-8b2e77.js   (a lazy chunk)
  assets/index-a1b2c3.css
```

How it works: the hash in each filename changes whenever the file's content changes. Browsers can cache hashed files forever because a new deploy produces new names; only `index.html` must be fresh, since it references the current names.

Stale chunks after a deploy: a user with an old `index.html` open may request a chunk that no longer exists. Vite emits an event you can handle:

```
// Fired when a dynamic import fails to load a preloaded
// chunk → reload to get the new index.html
window.addEventListener('vite:preloadError', () =>
  window.location.reload(),
);
```

16.2 Firebase Hosting (simplest Google option)

```
{
  "hosting": {
    "public": "dist",
    "ignore": ["firebase.json", "**/.*"],
    "rewrites": [
      {
        "source": "/api/**",
        "run": {
          "serviceId": "api",
          "region": "europe-west1"
        }
      },
      { "source": "**", "destination": "/index.html" }
    ],
```

```
    "headers": [
      {
        "source": "/assets/**",
        "headers": [
          {
            "key": "Cache-Control",
            "value": "public, max-age=31536000, immutable"
          }
        ]
      },
      {
        "source": "/index.html",
        "headers": [
          { "key": "Cache-Control", "value": "no-cache" }
        ]
      }
    ]
  }
}
```

What each part does (`firebase.json`):

- `public` : the folder to upload (Vite's output).
- First rewrite: requests to `/api/**` are proxied to a Cloud Run service — same origin as the app, so no CORS and cookies just work.
- Second rewrite: every other path serves `index.html` (SPA fallback, so `/projects/42` works on refresh). Rewrites are checked in order; real files in `dist` are served before rewrites apply.
- `headers` : long caching for hashed assets, revalidation for `index.html` .

```
# build, then upload dist/ atomically
npm run build && firebase deploy --only hosting
# temporary preview URL for a pull request
firebase hosting:channel:deploy pr-123 --expires 7d
```

16.3 Docker + Nginx (for Cloud Run / Kubernetes)

```
# Stage 1: build the static files with Node
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
```

```
# install exactly what the lockfile says (cached unless
# the package files change)
RUN npm ci
COPY . .
# build-time variable: VITE_* values are baked into the
# JS bundle
ARG VITE_API_URL
RUN npm run build

# Stage 2: a tiny web server image with only the built
# files (no Node, no source code)
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
# Cloud Run's default port
EXPOSE 8080
```

```
server {
  listen 8080;
  root /usr/share/nginx/html;

  # Hashed assets: cache for a year
  location /assets/ {
    add_header Cache-Control
      "public, max-age=31536000, immutable";
  }

  # Everything else: serve the file if it exists,
  # otherwise index.html (SPA fallback)
  location / {
    try_files $uri /index.html;
    add_header Cache-Control "no-cache";
  }

  gzip on;
  gzip_types text/css application/javascript
             application/json image/svg+xml;
}
```

Because `VITE_*` values are compiled in, one image = one environment. To promote the same image from staging to production, generate a small `/config.js` at container start (from environment variables) and read `window.__CONFIG__` in the app instead.

16.4 CI pipeline (GitHub Actions)

```
name: web
# run on every PR and every push
on: [pull_request, push]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        # caches ~/.npm between runs
        with: { node-version: 22, cache: npm }
      - run: npm ci
      # --run: single run, no watch mode
      - run: |
          npm run typecheck
          npm run lint
          npm test -- --run
      - run: |
          npx playwright install --with-deps chromium
          npm run build
          npm run test:e2e
      # keep the Playwright report to debug failures
      - uses: actions/upload-artifact@v4
        if: failure()
        with:
          name: playwright-report
          path: playwright-report
```

16.5 Error tracking and monitoring

```
// main.tsx
import * as Sentry from '@sentry/react';

Sentry.init({
  // public identifier, safe to ship
  dsn: env.VITE_SENTRY_DSN,
  // must match the release your source maps were
  // uploaded under
  release: import.meta.env.VITE_RELEASE,
  integrations: [
```

```
    // performance traces: page loads, navigations, fetches
    Sentry.browserTracingIntegration(),
    // session replay with text masked (privacy)
    Sentry.replayIntegration({ maskAllText: true }),
  ],
  // trace 10% of sessions (cost control)
  tracesSampleRate: 0.1,
  // always keep the replay when an error happens
  replaysOnErrorSampleRate: 1.0,
});

// Route React's root-level error hooks into Sentry
createRoot(el, {
  onUncaughtError: Sentry.reactErrorHandler(),
  onCaughtError: Sentry.reactErrorHandler(),
});
```

Upload source maps in CI (so stack traces show your original code) and don't serve them publicly. Track: JS errors per release, Web Vitals at the 75th percentile, API error rates, and business events (signup, checkout).

16.6 Feature flags and safe rollout

```
// Flags fetched like any server state; cached 5 minutes
const { data: flags } = useQuery({
  queryKey: ['flags'],
  queryFn: fetchFlags,
  staleTime: 5 * 60_000,
});
// the old path stays available as the fallback
return flags?.newCheckout ? <CheckoutV2 /> : <Checkout />;
```

A flag service (Firebase Remote Config, LaunchDarkly, GrowthBook, Unleash) adds percentage rollouts, targeting, kill switches and A/B tests. Deploy the code dark, release it by flag, and delete the flag after the rollout completes.

16.7 Performance budgets

Enforce budgets in CI so regressions fail the build: initial JS under ~170 KB gzipped for a typical app, LCP under 2.5 s on mid-range mobile (Lighthouse CI), no single chunk over ~250 KB.

17. Monorepos for enterprise teams

Most enterprise JavaScript lives in a monorepo: one repository holding several apps (web, admin, mobile, desktop, API) and the packages they share (UI kit, schemas, API client, configs). The 2026 default stack is **pnpm workspaces + Turborepo**; **Nx** is the main alternative for very large organizations that want code generators and enforced boundaries. This chapter applies to every guide in the series; the DevOps guide covers the CI/CD side in more depth.

17.1 Why monorepos (and the costs)

Benefit	Cost
One pull request can change an app and the shared package it uses, atomically	CI must be smart: only build and test what changed
One version of React/TypeScript/ESLint everywhere	dependency upgrades touch many apps at once
Share code without publishing packages to npm	needs clear ownership and dependency rules
One toolchain, one lockfile, consistent conventions	Git and IDE performance at very large scale

A polyrepo still fits when teams are fully independent, release on unrelated schedules, or need strict access separation.

17.2 Tooling landscape

Tool	Role	Notes
pnpm workspaces	package manager that links local packages	strict (no "phantom" dependencies), fast, disk-efficient; `catalog:` for shared versions
npm / Yarn / Bun workspaces	same role	fine for small repos
Turborepo	task runner + caching	minimal config, local and remote cache, `turbo prune` for Docker

Tool	Role	Notes
Nx	task runner + project graph + generators	`affected` commands, module-boundary lint rules, plugins for Next/RN/Node
Changesets	versioning, changelogs, publishing	for packages published to npm
syncpack / pnpm catalogs	keep dependency versions aligned	prevents duplicate React versions

17.3 Layout

```
acme/
  apps/          # deployable; never imported by others
    web/         # Vite React SPA     (@acme/web)
    site/        # Next.js marketing  (@acme/site)
    mobile/      # Expo app           (@acme/mobile)
    desktop/     # Electron + React   (@acme/desktop)
    api/         # Node/Express       (@acme/api)
  packages/      # importable; never deployed alone
    ui/          # shared React components
    schemas/     # zod schemas + types for client
                 # and server
    api-client/  # typed fetchers + query options
    utils/       # pure TS helpers
    config-ts/   # base tsconfig files
    config-eslint/  # shared ESLint flat config
  pnpm-workspace.yaml  # which folders are packages
  turbo.json     # task pipeline + caching rules
  package.json   # root: tooling scripts only
  .npmrc
  .github/CODEOWNERS   # who reviews which folder
```

17.4 Workspace configuration

```
# pnpm-workspace.yaml
# every folder matching these globs that has a
# package.json is a workspace package
packages:
  - 'apps/*'
  - 'packages/*'
# one place for shared dependency versions
catalog:
  react: ^19.2.0
  react-dom: ^19.2.0
  typescript: ^5.9.0
  zod: ^4.0.0
  '@tanstack/react-query': ^5.80.0
```

```
{
  "name": "acme",
  "private": true,
  "packageManager": "pnpm@10.x",
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "lint": "turbo run lint",
    "typecheck": "turbo run typecheck",
    "test": "turbo run test",
    "changeset": "changeset"
  },
  "devDependencies": {
    "turbo": "^2",
    "@changesets/cli": "^2",
    "typescript": "catalog:"
  }
}
```

The root `package.json` above holds tooling only. `private: true` prevents accidental publishing; `packageManager` pins the pnpm version (Corepack enforces it); each script delegates to Turborepo, which runs the task in every package that defines it; `"catalog:"` means "use the version from pnpm-workspace.yaml".

```
{
  "name": "@acme/web",
  "private": true,
  "dependencies": {
    "@acme/ui": "workspace:*",
    "@acme/api-client": "workspace:*",
    "react": "catalog:",
    "react-dom": "catalog:"
  }
}
```

In `apps/web/package.json`, `workspace:*` tells pnpm to link the local package (a symlink in node_modules) instead of downloading it from npm. Edits to `packages/ui` are visible in the app immediately.

17.5 Internal packages: source vs compiled

Just-in-time (source) packages export TypeScript files directly; each app's bundler compiles them. No build step, fastest feedback — the common choice for private packages.

```
{
  "name": "@acme/ui",
  "private": true,
  "type": "module",
  "exports": {
    "./button": "./src/button.tsx",
    "./dialog": "./src/dialog.tsx",
    "./styles.css": "./src/styles.css"
  },
  "peerDependencies": { "react": "catalog:" },
  "devDependencies": {
    "@acme/config-ts": "workspace:*",
    "react": "catalog:"
  }
}
```

In `packages/ui/package.json`: `exports` is the package's public API — `import { Button } from '@acme/ui/button'` works, deep imports of other files are blocked. React is a **peer** dependency, meaning "use the app's React": two copies of React in one bundle break hooks ("Invalid hook call").

```
// next.config.ts — Next.js must be told to compile
// TypeScript coming from these workspace packages
const config: NextConfig = {
  transpilePackages: ['@acme/ui', '@acme/api-client'],
};
// Vite and Expo (Metro) compile workspace TypeScript
// automatically.
```

Compiled packages build to `dist/` (with `tsdown` or `tsup`) and export JS plus `.d.ts` type files. Required when publishing to npm or when Node consumes the package without a bundler.

```
{
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.js"
    }
  },
  "scripts": {
    "build": "tsdown src/index.ts --dts --format esm",
    "dev": "tsdown --watch"
  }
}
```

Here `types` points TypeScript at the declarations and `import` points ESM consumers at the compiled JS; `--dts` emits the type declarations.

Prefer explicit subpath `exports` over a barrel `index.ts` that re-exports everything: barrels make bundlers, tests and the TypeScript server load unrelated code and slow everything down.

17.6 Shared TypeScript and ESLint config

```
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "verbatimModuleSyntax": true,
    "noUncheckedIndexedAccess": true,
    "skipLibCheck": true,
    "isolatedModules": true
  }
}
```

That's `packages/config-ts/base.json`, the shared baseline every package extends. Each app then extends a preset and adds only what's specific to it:

```
{
  "extends": "@acme/config-ts/react.json",
  "compilerOptions": { "paths": { "@/*": ["./src/*"] } },
  "include": ["src"]
}
```

```
// packages/config-eslint/react.js
// shared rules, versioned once
export default [
  ...base,
  reactHooks.configs['recommended-latest'],
  jsxA11y.flatConfigs.recommended,
];

// apps/web/eslint.config.js
// import the shared config, add local tweaks
import react from '@acme/config-eslint/react';
export default [...react, { ignores: ['dist'] }];
```

Avoid root-level `paths` aliases that point into other packages (`@acme/ui` → `../../packages/ui/src`): they bypass the package's `exports` and its declared dependencies. Let the package manager link packages.

17.7 Turborepo pipeline

```
{
  "$schema": "https://turborepo.com/schema.json",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "inputs": ["$TURBO_DEFAULT$", ".env*"],
      "outputs": [
        "dist/**",
        ".next/**",
        "!.next/cache/**"
      ],
      "env": ["VITE_API_URL", "NEXT_PUBLIC_*"]
    },
    "typecheck": { "dependsOn": ["^typecheck"] },
    "lint": {},
    "test": {
      "dependsOn": ["^build"],
      "outputs": ["coverage/**"]
    },
    "dev": { "cache": false, "persistent": true }
  }
}
```

How to read `turbo.json`:

- `"^build"` — the caret means "first run `build` in the packages this package depends on".
- `inputs` — files whose content forms the cache key (default: all tracked files of the package) plus env files.
- `outputs` — what to store in the cache and restore on a hit (Next's own cache folder excluded).
- `env` — environment variables that change the output; they're part of the cache key, otherwise a staging build could be served from a production cache.
- `lint: {}` — no dependencies, cached by default.
- `dev` — long-running servers: never cached, `persistent` tells Turbo they don't exit.

```
# everything; unchanged packages restore from cache
# in milliseconds
turbo run build
# one package
turbo run build --filter=@acme/web
# the package plus everything it depends on
turbo run build --filter=@acme/web...
# ui plus everything that depends ON it
# (who could break?)
turbo run test --filter=...@acme/ui
# only packages changed compared with main
turbo run lint test --affected
# run two dev servers together
turbo run dev --filter=@acme/web --filter=@acme/api
# enable the remote cache shared by team and CI
npx turbo login && npx turbo link
```

How caching works: Turbo hashes a task's inputs (files, env vars, dependency outputs). If the hash was seen before — locally or in the remote cache — it replays the logs and restores the outputs instead of running the task. Most cache misses come from undeclared env vars or outputs.

17.8 Nx equivalents

```
npx create-nx-workspace@latest acme
# interactive project dependency graph
nx graph
# run targets only for projects your changes affect
nx affected -t lint test build
# generator: scaffold a library with conventions
nx g @nx/react:lib ui --directory=packages/ui
```

```
// Enforce architecture with project tags
// (eslint.config.js)
'@nx/enforce-module-boundaries': [
  'error',
  {
    depConstraints: [
      // apps may only import features, UI and utils
      {
        sourceTag: 'type:app',
```

```
        onlyDependOnLibsWithTags: [
          'type:feature',
          'type:ui',
          'type:util',
        ],
      },
      // UI libraries can't import features (keeps the
      // design system generic)
      {
        sourceTag: 'type:ui',
        onlyDependOnLibsWithTags: ['type:util'],
      },
      // the admin area can't be imported by other scopes
      {
        sourceTag: 'scope:admin',
        onlyDependOnLibsWithTags: [
          'scope:admin',
          'scope:shared',
        ],
      },
    ],
  },
],
```

Pick Nx when you need generators, strict boundary enforcement and distributed task execution across hundreds of projects; pick Turborepo for minimal configuration on top of plain package-manager workspaces.

17.9 Sharing code across web, mobile, desktop and API

```
// packages/schemas/src/project.ts
// one schema used by every runtime
export const ProjectSchema = z.object({
  id: z.string().uuid(),
  name: z.string().min(1).max(80),
  status: z.enum(['active', 'archived']),
});
export type Project = z.infer<typeof ProjectSchema>;

// apps/api: validates request bodies with ProjectSchema
// apps/web + apps/mobile: validate API responses and
//   forms with the SAME schema
// packages/api-client: exports queryOptions used
//   identically by web, mobile and desktop
```

Share logic, types, schemas, API clients and hooks freely. Share UI only where the platform matches: web and Electron share DOM components; React Native needs its own primitives (or a cross-platform kit like Tamagui / react-native-web).

React Native notes: Expo detects monorepos and configures Metro automatically; keep exactly **one** `react` and `react-native` version in the repo (catalog); if a native library fails to resolve under pnpm's isolated layout, set `node-linker=hoisted` in `.npmrc`.

17.10 Ownership, boundaries, hygiene

```
# .github/CODEOWNERS
# PRs touching these paths need review from the team
/apps/web/           @acme/web-team
/apps/mobile/        @acme/mobile-team
/packages/ui/        @acme/design-system
/packages/schemas/   @acme/platform
```

- Enforce dependency direction (apps → features → ui/utils) with Nx tags or `eslint-plugin-boundaries`.
- No circular dependencies between packages (`madge --circular` or the Nx graph).
- Every package declares what it imports — pnpm's strictness surfaces "phantom" dependencies that only worked because another package installed them.
- One lockfile; review lockfile diffs; group shared upgrades in Renovate by catalog.
- "Invalid hook call" usually means two React copies — find them with `pnpm why react`.

17.11 Versioning and publishing (when packages leave the repo)

```
# interactive: pick changed packages, bump type
# (patch/minor/major), write a summary
pnpm changeset
# applies the bumps to package.json files and writes
# CHANGELOG.md entries
pnpm changeset version
# publishes the bumped packages to npm
pnpm changeset publish
```

Private apps don't need versions; only publishable packages (`"private": false`) do. The Changesets GitHub Action automates this by opening a "Version Packages" pull request.

17.12 CI and Docker in a monorepo

```
# .github/workflows/ci.yml
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        # full history: --affected compares against main
        with: { fetch-depth: 0 }
      # installs the pnpm version from packageManager
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: pnpm }
      # fail if the lockfile is out of date
      - run: pnpm install --frozen-lockfile
      - run: >-
          pnpm turbo run lint typecheck test build
          --affected
        env:
          # remote cache credentials
          TURBO_TOKEN: ${{ secrets.TURBO_TOKEN }}
          TURBO_TEAM: acme
```

```
# Build only what apps/api needs, not the whole monorepo
FROM node:22-alpine AS prune
WORKDIR /repo
RUN corepack enable
```

```
COPY . .
# Produces out/json (only the needed package.json files)
# and out/full (only the needed source)
RUN pnpm dlx turbo prune @acme/api --docker

FROM node:22-alpine AS build
WORKDIR /repo
RUN corepack enable
COPY --from=prune /repo/out/json/ .
# cached until a relevant package.json changes
RUN pnpm install --frozen-lockfile
COPY --from=prune /repo/out/full/ .
RUN pnpm turbo run build --filter=@acme/api

FROM node:22-alpine
WORKDIR /app
COPY --from=build /repo .
CMD ["node", "apps/api/dist/server.js"]
```

Deploy per app: each app has its own pipeline trigger (path filters or `--affected`), its own image and its own release cadence, even though the code lives together.

17.13 Advanced questions

1. When would you choose a polyrepo over a monorepo?
2. Source (just-in-time) vs compiled internal packages: trade-offs, and which needs `transpilePackages`?
3. Why do barrel files hurt in monorepos, and what replaces them?
4. What goes into a Turborepo cache key, and why do builds miss the cache?
5. How do you guarantee a single React version across web, mobile and a shared UI package?
6. How do you stop the mobile app from importing server-only code from `apps/api`?
7. Design CI that runs in under 10 minutes for a 60-package repo.

18. Capstone: TaskFlow, start to finish

TaskFlow is a team task manager: authentication, projects, a kanban board with drag-and-drop, filters in the URL, optimistic updates and real-time sync. It exercises every chapter; build it in the order below, committing after each step.

18.1 Step 1 — Requirements to architecture

Requirement	Decision	Why
Login, session persists	HttpOnly cookie session from the backend; a `useSession` query	tokens never reachable from JS (ch. 15.3)
Projects list with search/filter	URL search params + TanStack Query	shareable links, cached results (ch. 8.5, 9)
Board with columns, drag tasks	dnd-kit + an optimistic `moveTask` mutation	instant feedback, rollback on failure
Task detail drawer, deep-linkable	nested route `/projects/:id/tasks/:taskId`	URL opens the drawer directly
Live updates from teammates	Server-Sent Events → cache updates	one source of truth: the query cache
Theme + density preferences	persisted Zustand store	small global client state (ch. 8.2)

```
src/features/
  auth/      api.ts hooks.ts LoginPage.tsx
             RequireAuth.ts
  projects/  api.ts ProjectsPage.tsx ProjectCard.tsx
  board/     api.ts BoardPage.tsx Column.tsx
             TaskCard.tsx useMoveTask.ts
             useBoardEvents.ts
  tasks/     api.ts TaskDrawer.tsx TaskForm.tsx
  prefs/     store.ts
```

18.2 Step 2 — Domain types and API layer

```
// features/board/api.ts
// The schema is the single definition of a Task: runtime
// validation + TypeScript type.
export const TaskSchema = z.object({
  id: z.string(),
  projectId: z.string(),
  title: z.string(),
  status: z.enum(['todo', 'doing', 'done']),
  // fractional ordering within a column
```

```
  // (see positionBetween)
  position: z.number(),
  assigneeId: z.string().nullable(),
});
export type Task = z.infer<typeof TaskSchema>;
// 'todo' | 'doing' | 'done'
export type Status = Task['status'];

export const boardKeys = {
  board: (projectId: string) =>
    ['board', projectId] as const,
};

export const boardQuery = (projectId: string) =>
  queryOptions({
    queryKey: boardKeys.board(projectId),
    queryFn: ({ signal }) =>
      http<unknown>(`/projects/${projectId}/tasks`, {
        signal,
      }).then((d) => z.array(TaskSchema).parse(d)),
    // live updates arrive via SSE, so refetching rarely
    // is fine
    staleTime: 15_000,
  });

type Move = { id: string; status: Status; position: number };

export const moveTask = (t: Move) =>
  http<Task>(`/tasks/${t.id}/move`, {
    method: 'POST',
    json: { status: t.status, position: t.position },
  });
```

18.3 Step 3 — Routes

```
{
  // everything below requires a session
  Component: ProtectedLayout,
  loader: requireAuth,
  children: [
    {
      path: 'projects',
```

```
      Component: ProjectsPage,
      loader: () =>
        qc.ensureQueryData(projectQueries.list({})),
    },
    {
      path: 'projects/:projectId',
      Component: BoardPage,
      // board data before render
      loader: ({ params }) =>
        qc.ensureQueryData(boardQuery(params.projectId!)),
      // the drawer renders in BoardPage's <Outlet/>
      children: [
        { path: 'tasks/:taskId', Component: TaskDrawer },
      ],
    },
  ],
}
```

18.4 Step 4 — Board with grouped columns

```
const COLUMNS: Status[] = ['todo', 'doing', 'done'];
const byPosition = (a: Task, b: Task) =>
  a.position - b.position;

export function BoardPage() {
  const { projectId } = useParams() as {
    projectId: string;
  };
  // cached by the loader
  const { data: tasks } = useSuspenseQuery(
    boardQuery(projectId),
  );
  const [params] = useSearchParams();
  // optional filter from the URL
  const assignee = params.get('assignee');
  const move = useMoveTask(projectId);
  // live updates (step 6)
  useBoardEvents(projectId);

  const visible = assignee
    ? tasks.filter((t) => t.assigneeId === assignee)
    : tasks;
```

```
  // { todo: Task[], doing: Task[], done: Task[] }
  const byStatus = Object.groupBy(visible, (t) => t.status);
  const column = (s: Status) =>
    (byStatus[s] ?? []).toSorted(byPosition);

  // dnd-kit calls this when a drag ends; `over` is the
  // drop target (undefined if dropped outside)
  const onDragEnd = ({ active, over }: DragEndEvent) => {
    if (!over) return;
    // set by each droppable slot
    const target = over.data.current as {
      status: Status;
      index: number;
    };
    const col = column(target.status);
    move.mutate({
      id: String(active.id),
      status: target.status,
      position: positionBetween(col, target.index),
    });
  };

  // KeyboardSensor makes drag-and-drop usable without a
  // mouse (accessibility)
  const sensors = useSensors(
    useSensor(PointerSensor),
    useSensor(KeyboardSensor),
  );

  return (
    <DndContext onDragEnd={onDragEnd} sensors={sensors}>
      <div className="grid grid-cols-3 gap-4">
        {COLUMNS.map((s) => (
          <Column key={s} status={s} tasks={column(s)} />
        ))}
      </div>
      {/* the task drawer, when its route matches */}
      <Outlet />
    </DndContext>
  );
}

// Fractional positions: a dropped task gets the midpoint
```

```
// between its neighbors' positions, so only ONE row changes
// on the server (no renumbering of the whole column).
function positionBetween(col: Task[], index: number) {
  // neighbor above (or 0 at the top)
  const before = col[index - 1]?.position ?? 0;
  // neighbor below (or a gap at the bottom)
  const after = col[index]?.position ?? before + 2;
  return (before + after) / 2;
}
```

18.5 Step 5 — Optimistic move

```
export function useMoveTask(projectId: string) {
  const qc = useQueryClient();
  const key = boardKeys.board(projectId);
  return useMutation({
    mutationFn: moveTask,
    onMutate: async (m) => {
      // stop a refetch from overwriting our change
      await qc.cancelQueries({ queryKey: key });
      // snapshot for rollback
      const prev = qc.getQueryData<Task[]>(key);
      qc.setQueryData<Task[]>(key, (ts) =>
        ts?.map((t) =>
          t.id === m.id
            ? { ...t, status: m.status, position: m.position }
            : t,
        ),
      );
      return { prev };
    },
    // rollback
    onError: (_e, _m, ctx) => {
      qc.setQueryData(key, ctx?.prev);
      toast.error('Move failed');
    },
    // reconcile with the server
    onSettled: () =>
      qc.invalidateQueries({ queryKey: key }),
  });
}
```

18.6 Step 6 — Real-time sync

```
export function useBoardEvents(projectId: string) {
  const qc = useQueryClient();
  useEffect(() => {
    const key = boardKeys.board(projectId);
    // EventSource keeps one HTTP connection open; the
    // server pushes events as text
    const es = new EventSource(
      `/api/projects/${projectId}/events`,
      { withCredentials: true },
    );
    es.addEventListener('task.changed', (e) => {
      // validate pushed data too
      const task = TaskSchema.parse(JSON.parse(e.data));
      qc.setQueryData<Task[]>(key, (ts = []) => {
        const i = ts.findIndex((t) => t.id === task.id);
        // add new, or replace (ES2023 Array.with copies)
        return i === -1 ? [...ts, task] : ts.with(i, task);
      });
    });
    // EventSource reconnects automatically; events missed
    // meanwhile are recovered by refetching
    es.onerror = () =>
      qc.invalidateQueries({ queryKey: key });
    // close when leaving the board
    return () => es.close();
  }, [projectId, qc]);
}
```

18.7 Step 7 — Task drawer with form and deep link

```
export function TaskDrawer() {
  const { projectId, taskId } = useParams() as {
    projectId: string;
    taskId: string;
  };
  const navigate = useNavigate();
  const { data: tasks } = useSuspenseQuery(
    boardQuery(projectId),
  );
  const task = tasks.find((t) => t.id === taskId);
```

```
  // handled by the route ErrorBoundary
  if (!task) throw data('Task not found', { status: 404 });
  // keep active filters when closing
  const close = () =>
    navigate(`/projects/${projectId}${location.search}`);
  return (
    // Escape / overlay click → close
    <Sheet open onOpenChange={(o) => !o && close()}>
      {/* key: switching to another task remounts the */}
      {/* form with fresh default values */}
      <SheetContent>
        <TaskForm
          key={task.id}
          task={task}
          onSaved={close}
        />
      </SheetContent>
    </Sheet>
  );
}
```

18.8 Step 8 — Tests, then ship

- **Integration:** the board renders columns from MSW data; dragging with the keyboard sensor calls the move endpoint and reorders optimistically; a 500 response rolls back.
- **E2E:** login → create project → create task → move to Done → reload → still Done.
- **Ship:** CI green → Firebase preview channel per PR → merge → production deploy; Sentry release with source maps; new features behind flags.

18.9 Step 9 — Hardening pass

- [ ] Suspense + error boundaries per route and around the board
- [ ] Empty, loading, error and offline states designed
- [ ] Keyboard drag-and-drop and screen-reader announcements (dnd-kit `announcements`)
- [ ] Bundle analyzed; dnd-kit and the drawer lazy-loaded
- [ ] Lighthouse ≥ 90 on mobile; INP checked on a large board (virtualize columns over ~200 tasks)

19. Advanced checklist and interview-level questions

If you can answer each question below in two minutes with a code example, you operate at an advanced level in React.

19.1 Internals

1. What triggers a re-render? Why doesn't a prop change alone trigger one? (Parent render, state, context.)
2. Why must hooks be called in the same order? (Linked list on the fiber, indexed by call order.)
3. What's the difference between the render and commit phases, and which can be interrupted?
4. Why does `setCount(count + 1)` three times yield +1? How do updater functions fix it?
5. What does StrictMode double-invoke, and what bugs does that expose?
6. How does reconciliation use `type` and `key`? When would you deliberately change a key?

19.2 Hooks and effects

7. Give three cases where an effect is the wrong tool and the correct alternative for each.
8. How do you avoid race conditions in data fetching without a library?
9. When is `useLayoutEffect` required? What does `useSyncExternalStore` solve that `useEffect` + `useState` doesn't (tearing)?
10. What problem does `useEffectEvent` solve? Why can't you pass it to children?
11. Explain `useActionState`, `useOptimistic` and `useFormStatus` together in one form.

19.3 Performance

12. What does the React Compiler do, what are its requirements, and when do you still write `useMemo`?
13. Name three structural fixes that avoid memoization entirely.
14. Walk through diagnosing a slow interaction (INP 600 ms) from Profiler to fix.
15. How do transitions and `useDeferredValue` differ from debouncing?

19.4 Architecture

16. Classify state for a dashboard app and pick a tool for each category. Justify.
17. Why is Context a poor fit for high-frequency global state? What do selector-based stores do differently?
18. Design query keys and invalidation for projects → tasks → comments.
19. Design a reusable `<Combobox>` API: compound vs props, controlled vs uncontrolled, accessibility contract.
20. SPA (Vite) vs SSR framework (Next.js / React Router framework mode): what drives the decision? (SEO, TTFB/LCP, auth model, hosting, team.)

19.5 Quality and production

21. What do you test at each level of the testing trophy? Why mock the network rather than modules?
22. Where should auth tokens live in a browser app, and why?
23. How do you make a modal fully accessible? (Focus trap, return focus, Escape, `aria-modal`, labelled title, inert background.)
24. How do you roll out a risky feature safely and know whether it worked?
25. How do you handle a deploy that removes chunks a running tab still needs?

19.6 Advanced habits

- Write a short design note (problem, options, decision, trade-offs) before any change that touches more than one feature.
- Review PRs for behavior, accessibility and data flow — not just style; automate style.
- Keep dependencies few and current; upgrade React/Router/Query on a regular cadence using codemods.
- Leave every module more typed, more tested and simpler than you found it.
- Read the official docs at react.dev and the React blog's release posts; they are the source of truth for new APIs.

20. Cheat sheet

The code you write most often, with one-line explanations. Details live in the chapters referenced.

20.1 Components, JSX, TypeScript (ch. 3)

```
// Wrapper component: all native <button> props + our own
type ButtonProps = React.ComponentProps<'button'> & {
  variant?: 'primary' | 'ghost';
};
export function Button({
  variant = 'primary',
  ...rest
}: ButtonProps) {
  // forward everything else
  return <button data-variant={variant} {...rest} />;
}

// Named slots via props
function Card({
  title,
  children,
  footer,
}: {
  title: string;
  children: React.ReactNode;
  footer?: React.ReactNode;
}) {
  return (
    <section>
      <h2>{title}</h2>
      {children}
      {footer && <footer>{footer}</footer>}
    </section>
  );
}

// React 19: ref is a normal prop
function Input({
  ref,
  ...rest
}: React.ComponentProps<'input'>) {
  return <input ref={ref} {...rest} />;
}

// Generic component: T inferred from `items`
function List<T>({
```

```
  items,
  render,
  getKey,
}: {
  items: T[];
  render: (t: T) => React.ReactNode;
  getKey: (t: T) => React.Key;
}) {
  return (
    <ul>
      {items.map((i) => (
        <li key={getKey(i)}>{render(i)}</li>
      ))}
    </ul>
  );
}
```

```
// either/or
{isLoading ? <Spinner /> : <Content />}
// render only when truthy
{error && <ErrorBox error={error} />}
// NOT {count && …}: 0 would render as "0"
{count > 0 && <Badge n={count} />}
// stable id keys, never index for dynamic lists
{users.map((u) => <UserRow key={u.id} user={u} />)}
// keyed fragment inside map
<Fragment key={item.id}>…</Fragment>
<input onChange={(e) => setValue(e.target.value)} />
// stop the full-page form post
<form
  onSubmit={(e) => {
    e.preventDefault();
    submit();
  }}
/>
<div
  className={cn('p-4', active && 'bg-blue-50')}
  style={{ width: `${pct}%` }}
/>
// code-split; render inside <Suspense>
const Settings = lazy(() => import('./Settings'));
```

```
// render elsewhere in the DOM
createPortal(<Modal />, document.body);
// new key = fresh component state
<EditForm key={user.id} user={user} />
```

Type	Use for
`React.ReactNode`	anything renderable (children)
`React.ComponentProps<'button'>`	all native props of an element
`React.ComponentProps<typeof X>`	props of an existing component
`React.ChangeEvent<HTMLInputElement>`	input change events
`React.FormEvent<HTMLFormElement>`	form submit events
`React.Dispatch<React.SetStateAction<T>>`	a `useState` setter passed as a prop

20.2 Hooks (ch. 4–5)

```
const [v, setV] = useState('');
// lazy: runs only on first render
const [cfg] = useState(() => expensiveInit());
// updater: based on the latest value
setCount((c) => c + 1);
// add (new array → React sees the change)
setItems((xs) => [...xs, item]);
// remove
setItems((xs) => xs.filter((x) => x.id !== id));
// update one
setItems((xs) =>
  xs.map((x) =>
    x.id === id ? { ...x, done: !x.done } : x,
  ),
);

// complex transitions in one pure function
const [state, dispatch] = useReducer(reducer, initial);

// sync with something outside React
useEffect(() => {
  const id = setInterval(tick, 1000);
```

```
  // cleanup: stop what you started
  return () => clearInterval(id);
  // [] = no reactive dependencies
}, []);

// manual fetch with race protection
useEffect(() => {
  const ac = new AbortController();
  fetch(`/api/u/${id}`, { signal: ac.signal })
    .then((r) => r.json())
    .then(setUser)
    .catch(() => {});
  // id changed → cancel the stale request
  return () => ac.abort();
}, [id]);

// DOM handle; inputRef.current?.focus()
const inputRef = useRef<HTMLInputElement>(null);
// only without the React Compiler
const sorted = useMemo(
  () => items.toSorted(byDate),
  [items],
);
const ThemeCtx = createContext<'light' | 'dark'>('light');
// React 19 provider syntax
<ThemeCtx value="dark">{children}</ThemeCtx>
// read context (may be conditional)
const theme = use(ThemeCtx);
// a11y ids, stable across server/client
const id = useId();
// low-priority, interruptible updates
const [isPending, startTransition] = useTransition();
// lagging copy for heavy renders
const deferredQuery = useDeferredValue(query);
// latest values inside effects, not a dependency
const onMsg = useEffectEvent((m: Msg) =>
  notify(m, theme),
);
```

```
// React 19 form actions (ch. 4.10, 11.2)
const [state, formAction, pending] = useActionState(
  async (prev: State, fd: FormData) => {
    const r = await save(fd.get('name') as string);
    // becomes the new `state`
    return r.ok ? { ok: true } : { error: r.message };
  },
  {},
);
<form action={formAction}>
  <input name="name" />
  <button disabled={pending}>Save</button>
</form>
// inside a child of <form>
const { pending } = useFormStatus();
const [optimistic, addOptimistic] = useOptimistic(
  todos,
  (cur, t: Todo) => [...cur, t],
);
```

20.3 Server and client state (ch. 8–10)

```
// TanStack Query
export const todoQueries = {
  list: (f: Filters) =>
    queryOptions({
      queryKey: ['todos', 'list', f],
      queryFn: () => api.todos(f),
    }),
};
const { data, isPending, isError } = useQuery(
  todoQueries.list(filters),
);
// keep old results while new ones load
useQuery({
  ...todoQueries.list(f),
  placeholderData: keepPreviousData,
});
const m = useMutation({
  mutationFn: api.createTodo,
  onSuccess: () =>
    qc.invalidateQueries({ queryKey: ['todos'] }),
```

```
});
// in loaders: cached or fetch
await qc.ensureQueryData(todoQueries.list({}));

// Zustand (persisted to localStorage under 'app-store')
export const useStore = create<State>()(
  persist(
    (set) => ({
      count: 0,
      inc: () => set((s) => ({ count: s.count + 1 })),
    }),
    { name: 'app-store' },
  ),
);
// select a slice → minimal re-renders
const count = useStore((s) => s.count);
// several fields → shallow compare
const { inc } = useStore(
  useShallow((s) => ({ inc: s.inc })),
);

// React Router 7
const { id } = useParams();
const data = useLoaderData<typeof loader>();
// URL state
const [params, setParams] = useSearchParams();
const navigate = useNavigate();
navigate('/login', { replace: true });
// inside loaders/actions
throw redirect('/login');
```

20.4 Forms, styling, testing (ch. 11, 12, 14)

```
// React Hook Form + Zod
const {
  register,
  handleSubmit,
  formState: { errors, isSubmitting },
} = useForm<Values>({ resolver: zodResolver(Schema) });
// save receives valid, typed values
<form onSubmit={handleSubmit(save)}>
  <input
```

```
    {...register('email')}
    aria-invalid={!!errors.email}
  />
  {errors.email && (
    <p role="alert">{errors.email.message}</p>
  )}
</form>

// Tailwind class merging + variants
// (later classes win conflicts)
export const cn = (...c: ClassValue[]) =>
  twMerge(clsx(c));
const button = cva('rounded-md font-medium', {
  variants: {
    size: { sm: 'h-8 px-3', md: 'h-10 px-4' },
  },
});

// Testing Library + MSW
const user = userEvent.setup();
render(<LoginForm onSubmit={onSubmit} />);
await user.type(
  screen.getByLabelText(/email/i),
  'a@b.com',
);
await user.click(
  screen.getByRole('button', { name: /sign in/i }),
);
// findBy waits for async UI
expect(
  await screen.findByText(/welcome/i),
).toBeInTheDocument();
// per-test failure
server.use(
  http.get(
    '*/api/user',
    () => new HttpResponse(null, { status: 500 }),
  ),
);
```

20.5 Monorepo commands (ch. 17)

```
# add a dependency to one package
pnpm add zod --filter @acme/web
# link an internal package
pnpm add @acme/ui@workspace:* --filter @acme/web
# find duplicate versions ("Invalid hook call")
pnpm why react
# a package plus its dependencies
turbo run build --filter=@acme/web...
# only what changed vs main
turbo run lint test build --affected
# minimal subset for a Docker build
turbo prune @acme/api --docker
```

20.6 Common gotchas

Symptom	Cause	Fix
`0` rendered on screen	`{count && <X/>}`	`{count > 0 && <X/>}`
State "one behind" right after setState	state is a per-render snapshot	use the new value directly, or an updater function
Effect runs every render	object/function in dependencies	depend on primitives; build objects inside the effect
Infinite render loop	setState during render, or effect without correct deps	compute the value instead; fix deps
Input loses focus on each keystroke	component defined inside another component	define components at module level
List rows swap state	index used as key	stable id as key
Interval sees stale state	closure captured an old value	updater function, ref, or `useEffectEvent`
Hydration mismatch (SSR)	`Date.now()`, `Math.random()`, `window` during render	move to an effect, or render a stable server value

Symptom	Cause	Fix
"Cannot update a component while rendering another"	setState called in a child's render	move it to an effect or event handler
Effects run twice in development	StrictMode	write proper cleanup — expected behavior